

База знаний РЕД База Данных

- СУБД РЕД База Данных
 - [Описание продукта](#)
 - [Документация](#)
 - [Системные требования](#)
 - [Установка РЕД База Данных](#)
 - [Обновление СУБД РЕД База Данных](#)
 - [Диагностика проблем](#)
 - [Часто задаваемые вопросы по СУБД РЕД База Данных \(FAQ\)](#)
 - [Соответствие стандарту SQL:2023](#)

Описание продукта

СУБД РЕД База Данных (далее СУБД или РЕД База Данных) используется для упорядоченного хранения и обработки больших объемов информации. РЕД База Данных представляет собой мощную современную СУБД с открытым кодом. Ядро РЕД Базы Данных построено на основе одной из самых известных и распространенных в мире СУБД с открытым кодом – **Firebird**, которая используется в решениях различного масштаба: от встроенных аппаратных систем и решений для небольших компаний до IT-систем крупнейших транснациональных корпораций с размерами баз данных до десятков терабайт и десятками миллионов транзакций в день.

Назначение

СУБД РЕД База Данных предоставляет пользователям следующие возможности:

- поддержка всех основных платформ и операционных систем (среди них: Windows, семейство Linux, BSD Unix, IBM AIX, HP-UX, Sun Solaris и др.);
- поддержка 64-битных систем;
- поддержка многопроцессорных и многоядерных аппаратных платформ;
- высокое быстродействие;
- возможность хранения базы данных в одном отдельном файле;
- возможность аутентификации и авторизации пользователей с использованием в качестве источников сведений об учетных записях пользователей защищенной БД пользователей или системного каталога;
- возможность «горячего» резервного копирования БД и инкрементного резервного копирования, в т.ч. с применением аппаратных решений для резервного копирования;
- наличие модулей сопряжения практически для всех используемых сред разработки, результатов тестов этих модулей и гарантия стабильной работы;
- возможность работы во «встроенном» ПО (**embedded**) в локальном режиме в виде библиотеки **DLL** без отдельной установки и настройки СУБД РЕД База Данных, в том числе поддержка встраивания в виртуальную машину Java;
- возможность одновременной модификации базы данных несколькими пользователями;
- обратная совместимость с базами данных от предыдущих версий **Firebird**;
- многоверсионная архитектура;
- модульная архитектура;

- модульная архитектура,
- соответствие большинству требований стандарта ISO/ANSI SQL;
- низкие требования к аппаратному обеспечению для небольших баз данных;
- большие возможности по расширению функционала самой СУБД посредством модулей;
- ядро, изначально основанное на многоверсионной архитектуре (**MGA**);
- полное соответствие принципам атомарности, непротиворечивости, изоляции, долговечности (**ACID**).

Вы можете подробнее ознакомиться с установкой **СУБД РЕД База Данных**, а также пройти обучающий курс по работе с **СУБД РЕД База Данных**, просмотрев наши обучающие видео:

- на **RuTube**:
 - [Установка СУБД РЕД База Данных на РЕД ОС](#);
 - [Курс «Администрирование СУБД РЕД База Данных 3.0»](#).
- в **Яндекс.Дзен**:
 - [Установка СУБД РЕД База Данных на РЕД ОС](#);
 - [Курс «Администрирование СУБД РЕД База Данных 3.0»](#).

На наших каналах вы также сможете найти много другой полезной информации.

Документация

Наиболее полную, актуальную и регулярно дорабатываемую документацию можно найти [на сайте СУБД РЕД База Данных](#) в разделе «[Документация](#)».

Системные требования

Аппаратная часть

Программная часть

Аппаратная часть

Для установки и нормальной работы СУБД РЕД База Данных персональный компьютер должен быть оснащен комплектующими со следующими характеристиками:

- Процессор **Intel Pentium D** и выше, процессор **AMD** с поддержкой **x86_64** или процессор **Intel** с поддержкой набора инструкций **EM64T** для архитектуры **x86_64**.
- Оперативная память от **4 ГБ**.
- Запоминающее устройство объемом **не менее 21 Гб**.
- Клавиатура 101/102-х клавишная русская/латинская.
- Сетевая карта с поддержкой **Ethernet**.

Рекомендованные системные требования подбираются индивидуально в зависимости от размера базы данных, количества пользователей и характера нагрузки.

Программная часть

Для установки и эксплуатации СУБД в ОС семейства *Windows* не требуется установка дополнительного программного обеспечения.

Для установки и эксплуатации СУБД в ОС семейства Linux необходимы библиотеки: **glibc 2.17 и выше, libstdc++ от gcc 10.2 и выше.**

Установка РЕД База Данных

[Установка для Linux на примере РЕД ОС 8](#)

[Загрузка установочного файла на РЕД ОС 8](#)

[Последовательность установки СУБД на РЕД ОС 8](#)

[Установка для ОС Windows](#)

[Загрузка установочного файла на ОС Windows](#)

[Последовательность установки СУБД на ОС Windows](#)

Вы можете подробнее ознакомиться с установкой **СУБД РЕД База Данных**, а также пройти обучающий курс по работе с **СУБД РЕД База Данных**, просмотрев наши обучающие видео:

- на **RuTube**:
 - [Установка СУБД РЕД База Данных на РЕД ОС](#);
 - [Курс «Администрирование СУБД РЕД База Данных 3.0»](#).
- в **Яндекс.Дзен**:
 - [Установка СУБД РЕД База Данных на РЕД ОС](#);
 - [Курс «Администрирование СУБД РЕД База Данных 3.0»](#).

На наших каналах вы также сможете найти много другой полезной информации.

Установка для Linux на примере РЕД ОС 8

Загрузка установочного файла на РЕД ОС 8

Скачать РЕД База Данных можно [на официальном сайте](#) компании в меню **Продукты -> Загрузки**.

Для скачивания необходимо авторизоваться или зарегистрироваться, а также купить лицензию.

Для установки в РЕД ОС необходимо скачать бинарный файл нужной разрядности и назначить на него права:

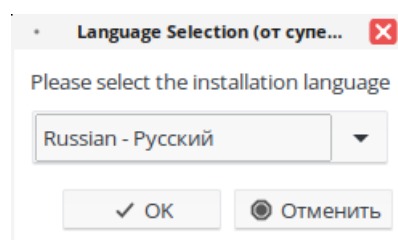
```
sudo chmod +x <путь_до_файла>
```

Для старта установки запустите установочный бинарный файл:

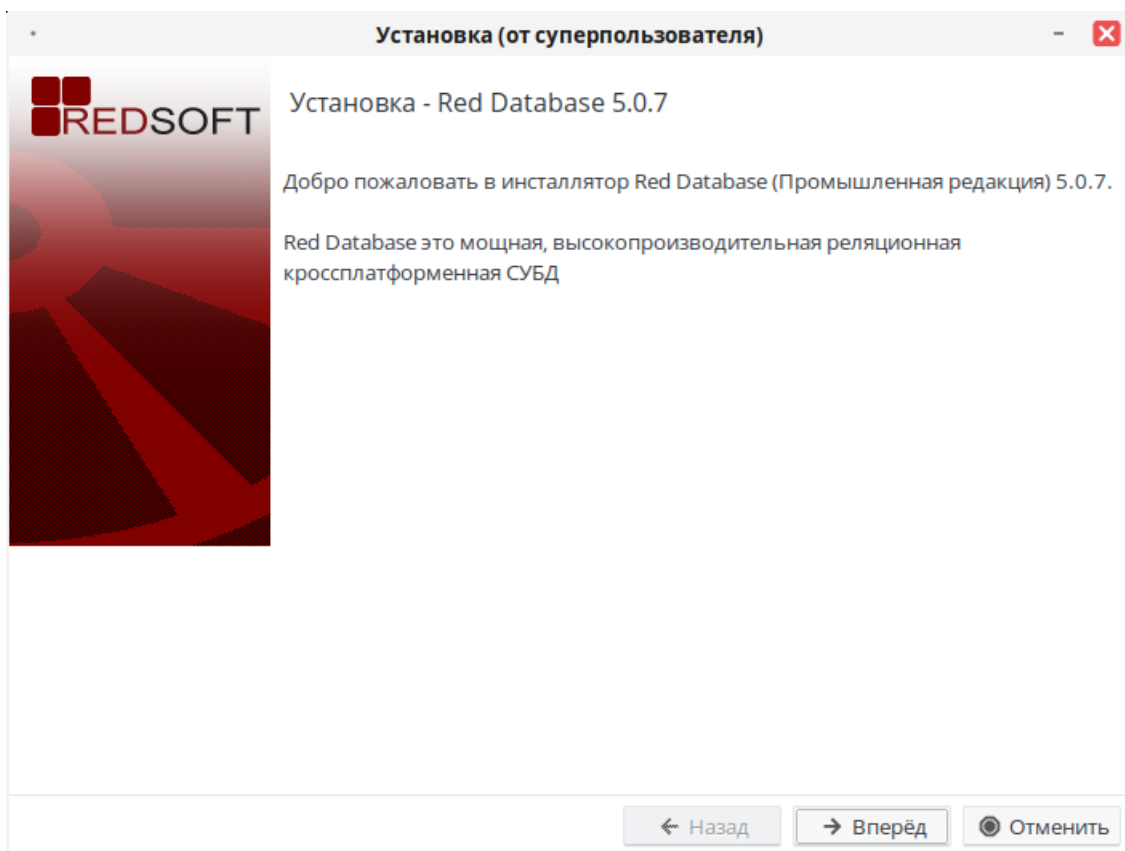
```
sudo ./rdb50-linux-x86_64-standart-5.0.7.bin
```

Последовательность установки СУБД на РЕД ОС 8

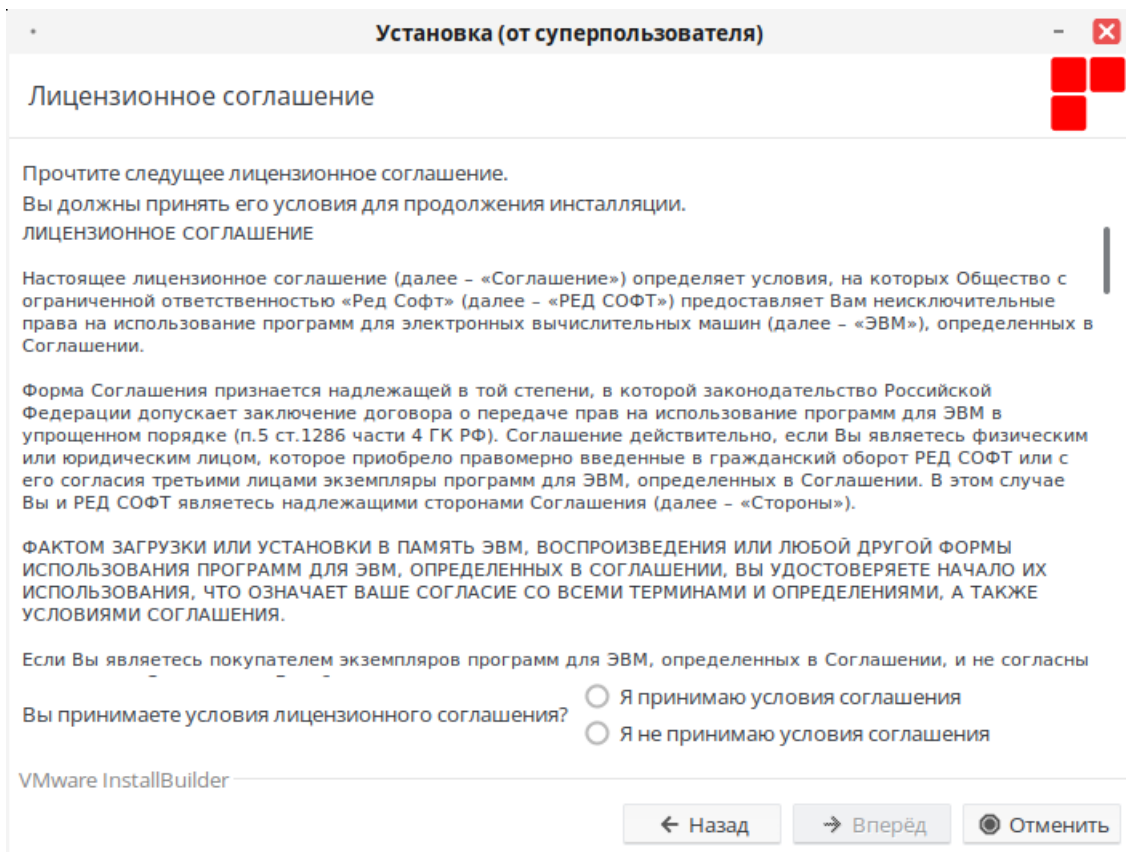
После запуска бинарного файла будет открыто окно настройки языка установщика. Выберите из списка необходимый язык и нажмите «**ОК**».



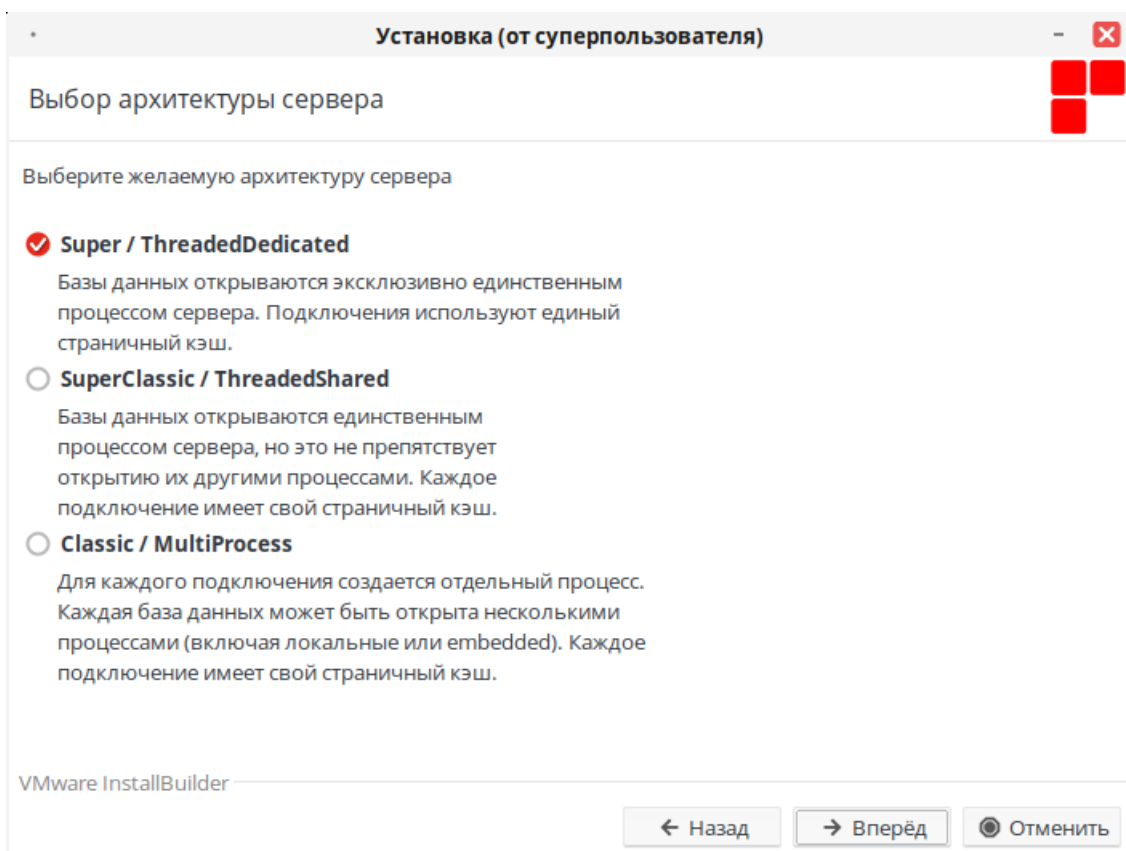
Будет открыто приветственное окно установщика. Нажмите «**Вперед**».



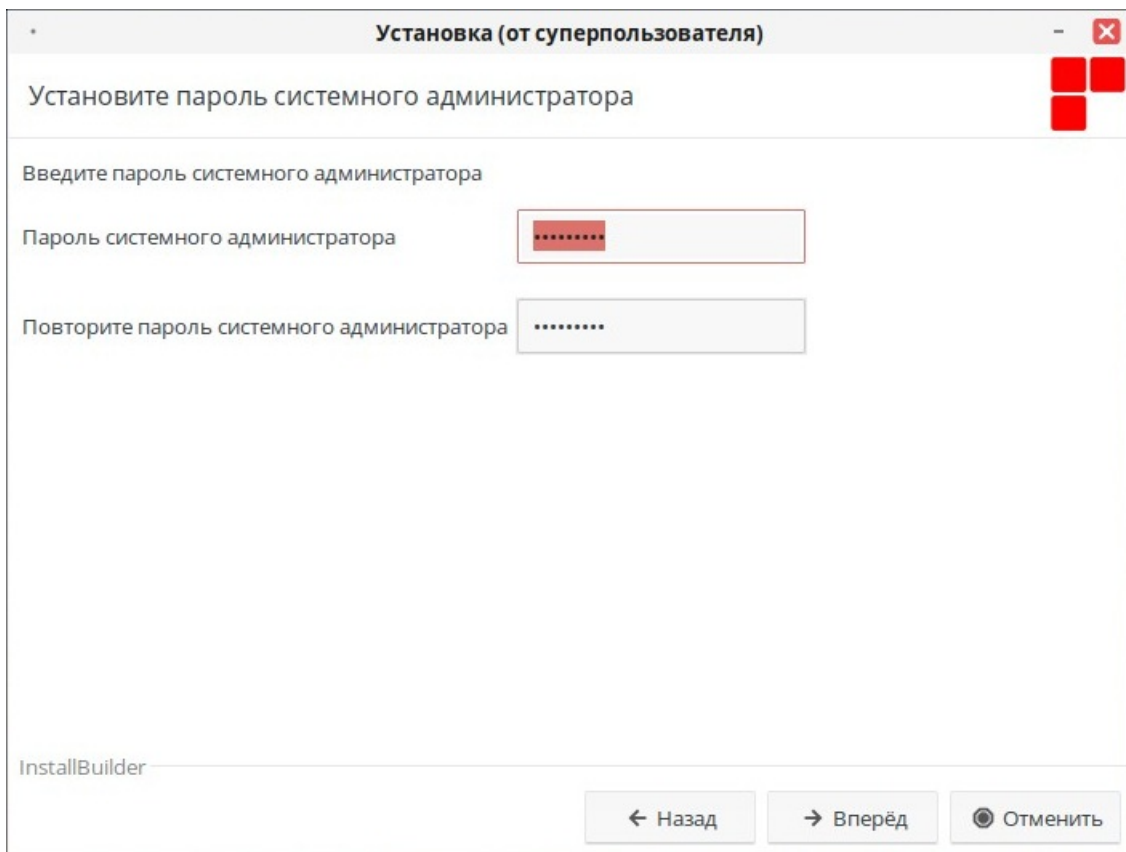
Далее прочтите и примите условия лицензионного соглашения, установив флаг для параметра «**Я принимаю условия соглашения**». Нажмите «**Вперед**».



Выберите требуемую архитектуру сервера, установив флаг для соответствующего параметра. Нажмите «**Вперед**».



Установите пароль системного администратора. Пароль необходимо повторить для избежания опечаток. Нажмите «**Вперед**».



Установка (от суперпользователя)

Установите пароль системного администратора

Введите пароль системного администратора

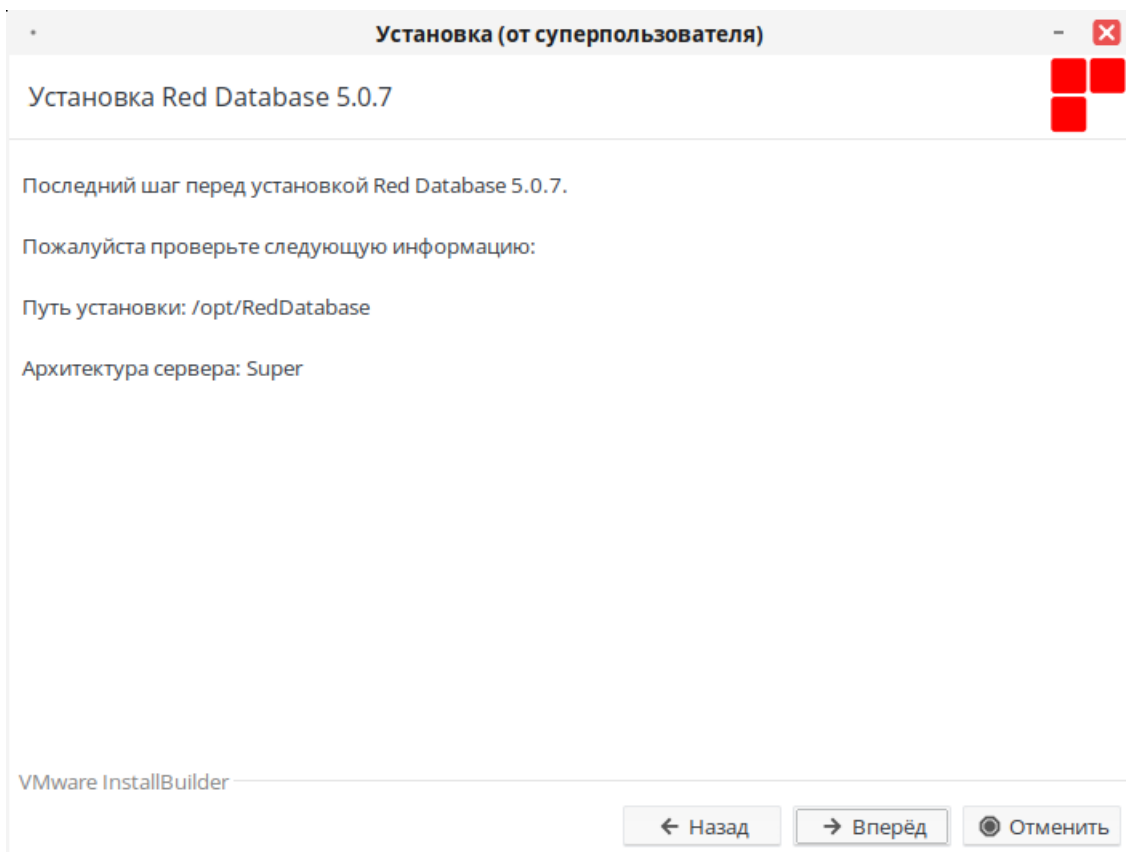
Пароль системного администратора

Повторите пароль системного администратора

InstallBuilder

← Назад → Вперёд ⌂ Отменить

Проверьте указанные ранее параметры. Если все настроено верно, нажмите «**Вперед**» для запуска процесса установки. В противном случае нажмите «**Назад**» и внесите изменения в необходимые конфигурации.



Установка (от суперпользователя)

Установка Red Database 5.0.7

Последний шаг перед установкой Red Database 5.0.7.

Пожалуйста проверьте следующую информацию:

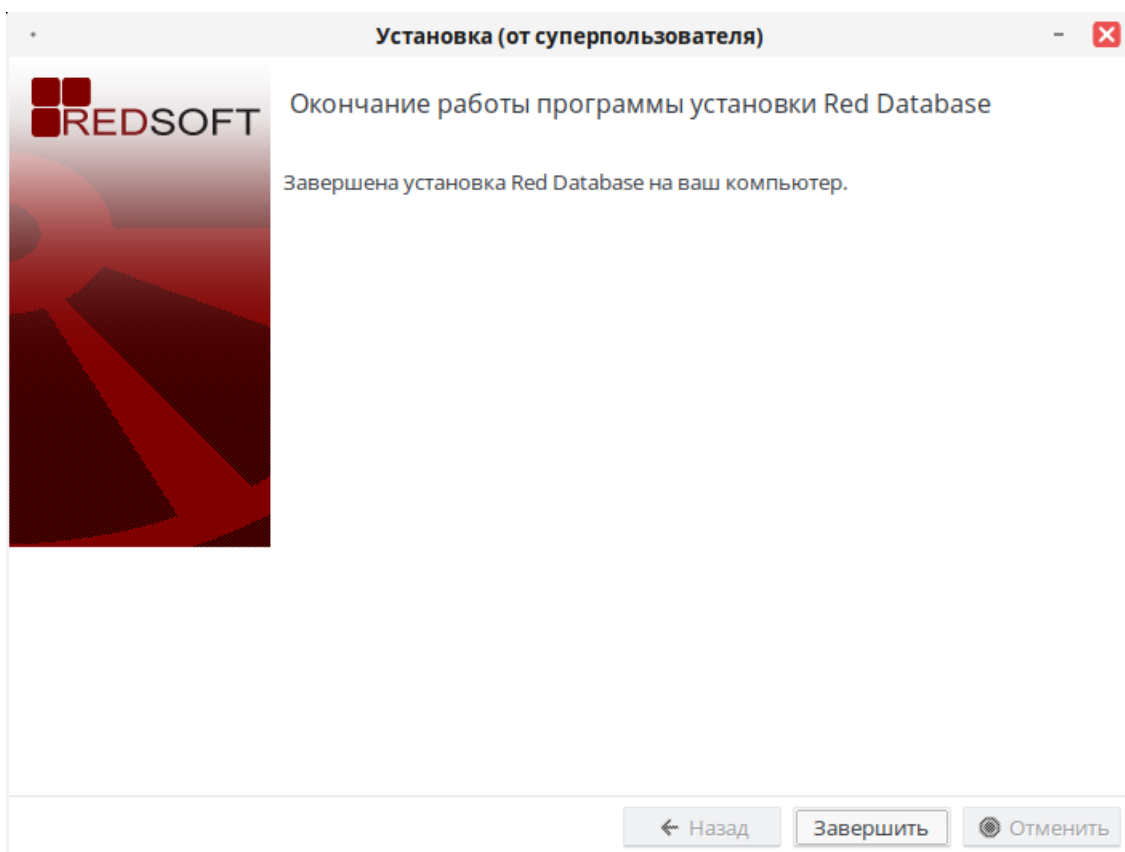
Путь установки: /opt/RedDatabase

Архитектура сервера: Super

VMware InstallBuilder

← Назад → Вперёд ⌂ Отменить

После завершения процесса установки будет выведено соответствующее сообщение. Нажмите «**Завершить**».



Установка СУБД 8 в текстовом режиме

Если программа инсталляции запущена в текстовом режиме (с ключом --mode text):

```
# ./RedDatabase-5.0.X.X-linux-X.bin --mode text
```

то она последовательно будет выводить запросы на подтверждение тех или иных параметров установки, таких как выбор компонентов или пароль пользователя SYSDBA. В случае, если на тот или иной запрос мастера установки предусмотрен ответ по умолчанию, то такой вариант обозначен заглавной буквой, и в этом случае этот вариант можно подтвердить нажатием клавиши «Ввод», в случае же, если выбор по умолчанию не предусмотрен, то необходимо обязательно ответить на вопрос «Да» (Y) или «Нет» (N).

```

Архитектура сервера

[1] Super / ThreadedDedicated: Базы данных открываются эксклюзивно единственным процессом
[2] SuperClassic / ThreadedShared: Базы данных открываются единственным процессом сервера
[3] Classic / MultiProcess: Для каждого подключения создается отдельный процесс. Каждая база
свой страничный кэш.
Пожалуйста, выберите опцию [1] : 1

-----
Установите пароль системного администратора

Введите пароль системного администратора (по умолчанию masterkey)

Пароль системного администратора [*****] :

-----
Установка Red Database (Стандартная редакция) 4.0.0.1711

Последний шаг перед установкой Red Database (Стандартная редакция) 4.0.0.1711.

Пожалуйста проверьте следующую информацию:

Путь установки: /opt/RedDatabase

Архитектура сервера: Super
Нажмите [Ввод] для продолжения :

-----
Пожалуйста, подождите пока программа установит Red Database (Стандартная
редакция) на ваш компьютер.

Установка
0% _____ 50% _____ 100%
#####

-----
Завершена установка Red Database (Стандартная редакция) на ваш компьютер.

```

Удаление сервера осуществляется, по аналогии с графическим режимом, запуском программы uninstall из каталога установки Ред База Данных.

Установка для ОС Windows

Загрузка установочного файла на ОС Windows

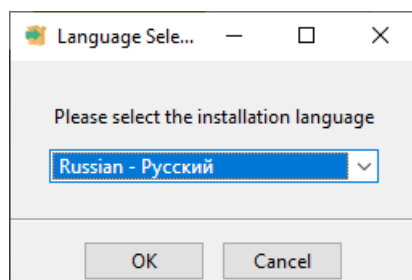
Скачать РЕД Базу Данных можно [на официальном сайте](#) компании в разделе «**Загрузки**».

Для скачивания необходимо авторизоваться или зарегистрироваться, а также купить лицензию

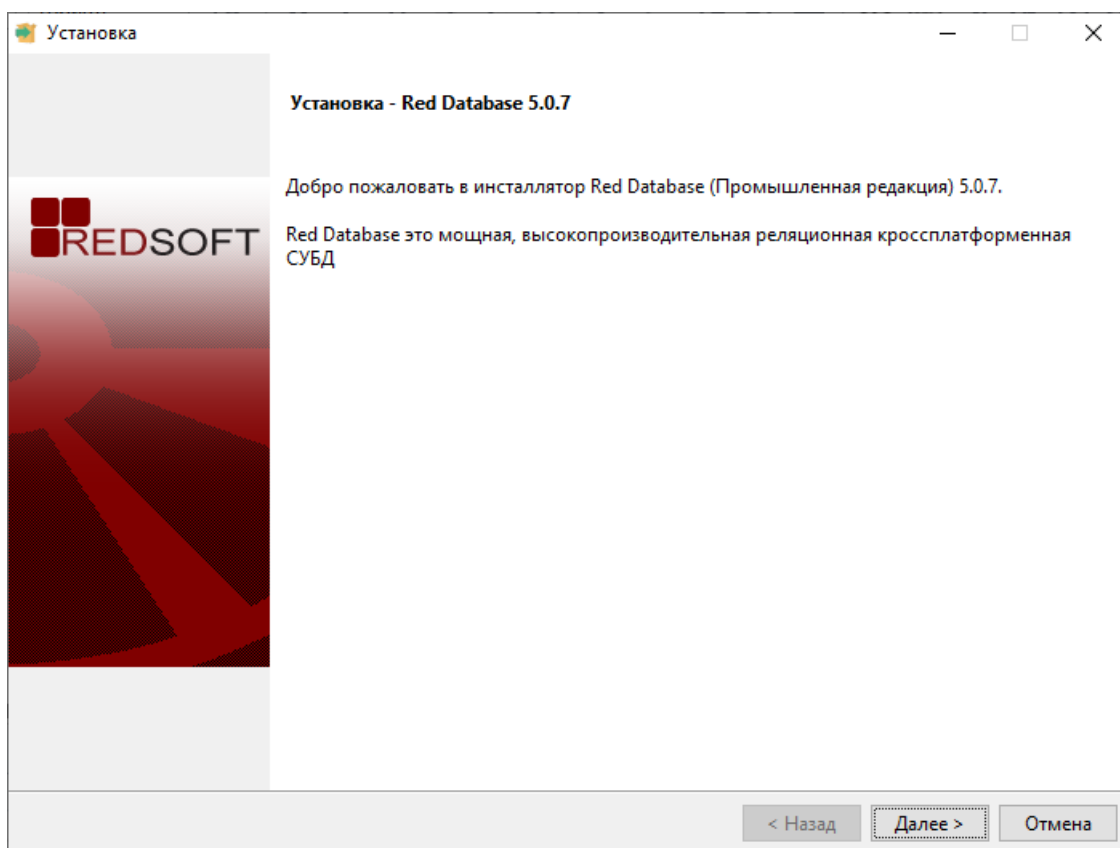
Для установки на Windows скачайте файл с расширением **.exe** нужной архитектуры. Далее от имени администратора запустите установщик.

Последовательность установки СУБД на ОС Windows

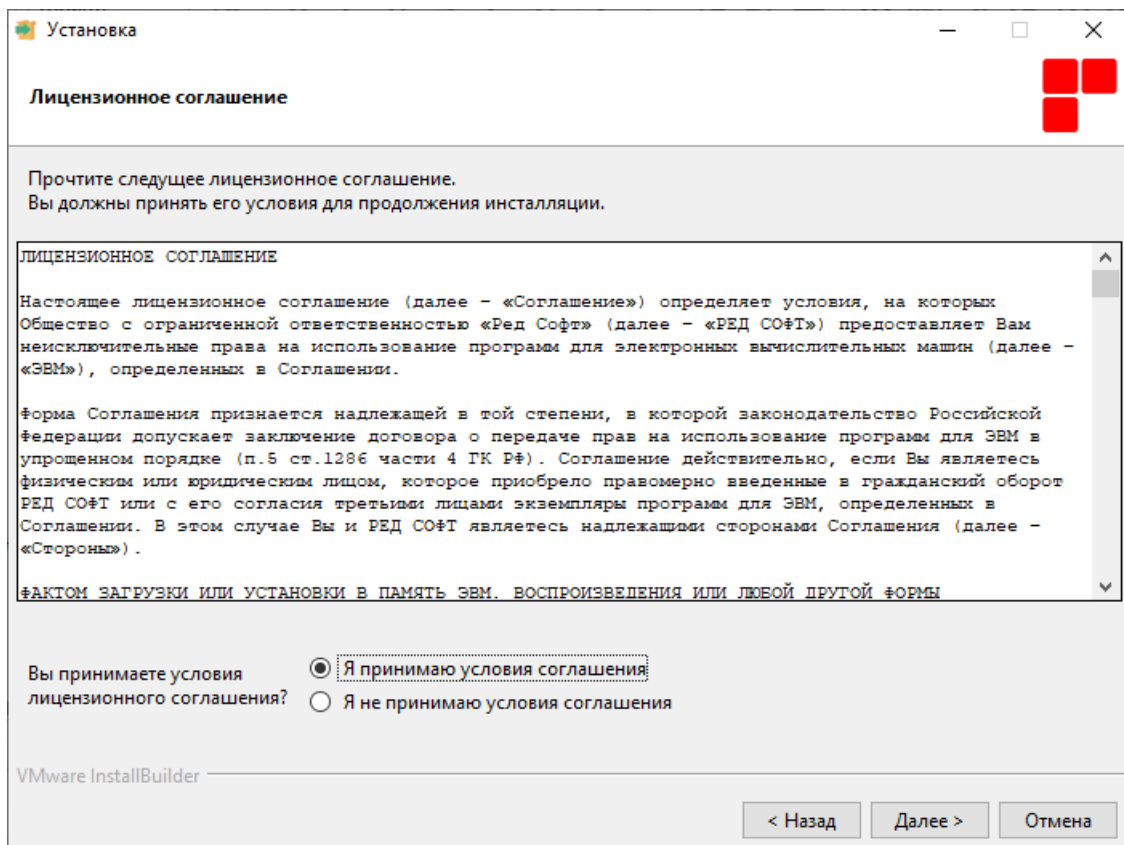
После запуска **exe**-файла будет открыто окно настройки языка установщика. Выберите из списка необходимый язык и нажмите «**ОК**».



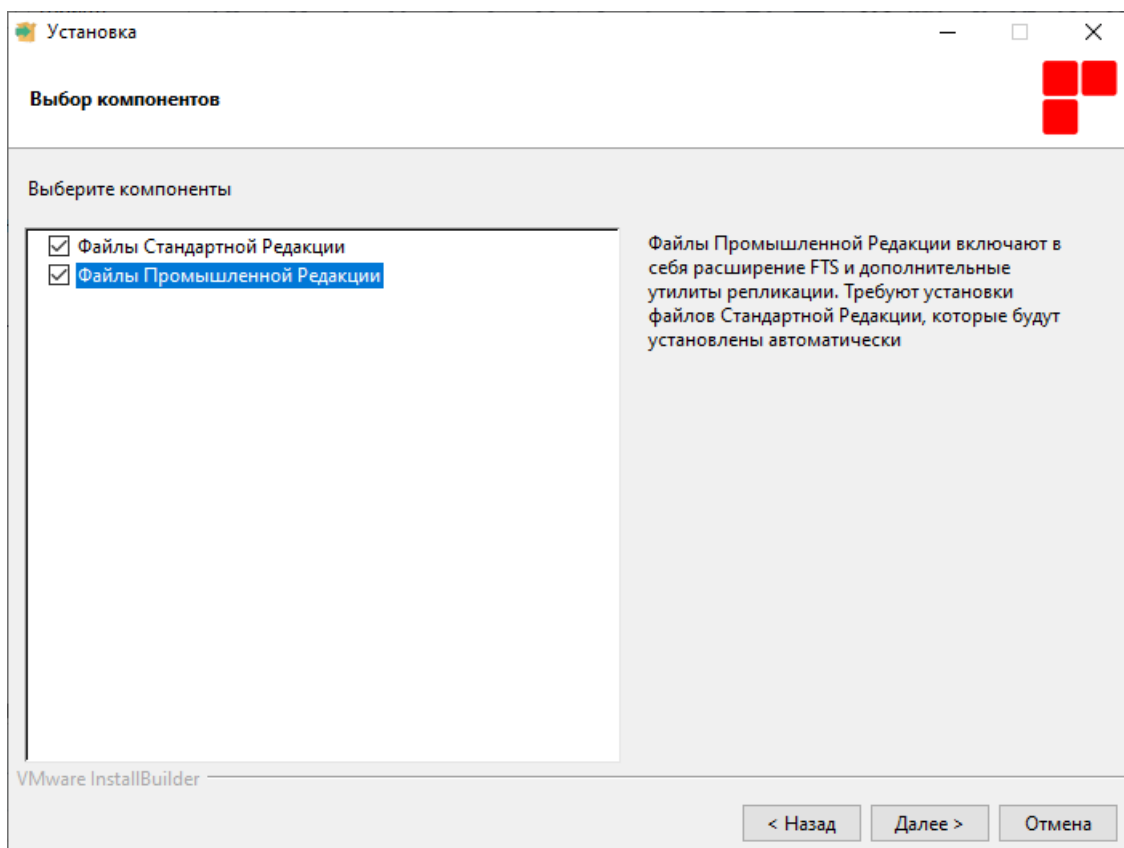
Будет открыто приветственное окно установщика. Нажмите «**Далее**».



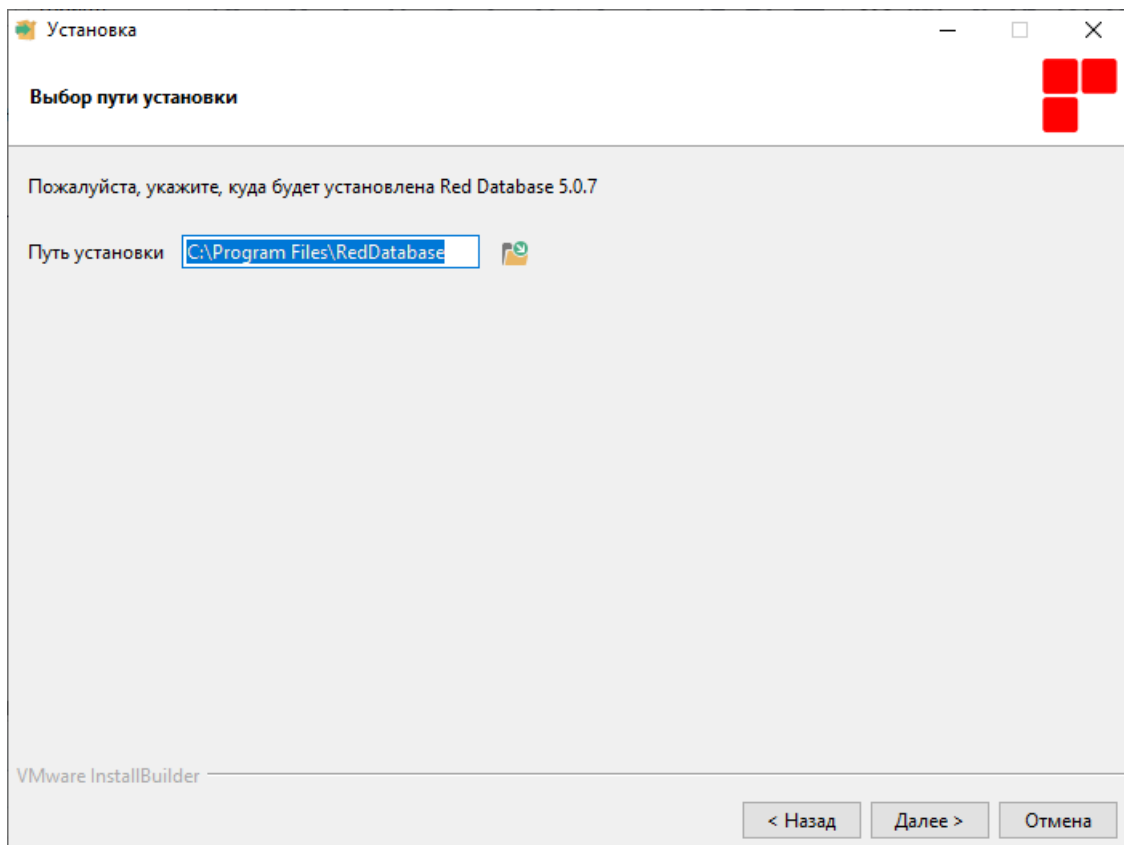
Далее прочтите и примите условия лицензионного соглашения, установив флаг для параметра «**Я принимаю условия соглашения**». Нажмите «**Далее**».



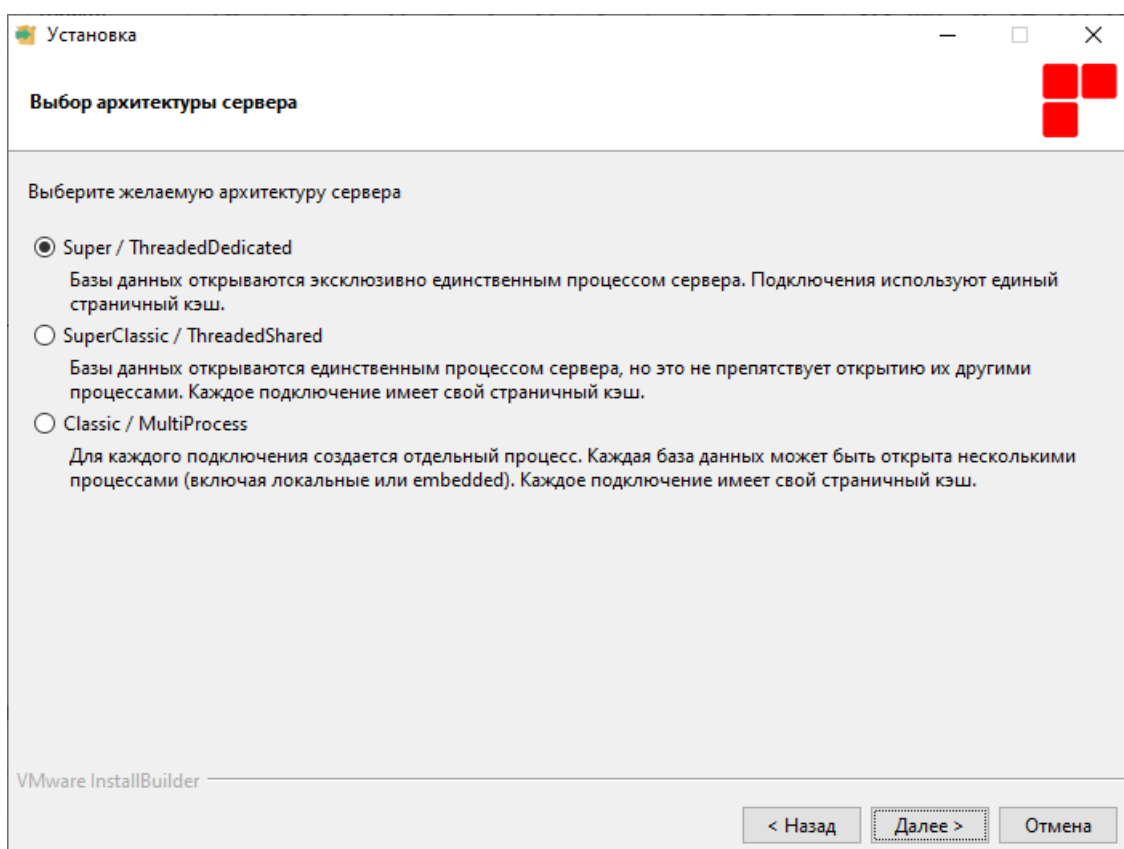
Выберите необходимые компоненты. Нажмите «Далее».



Выберите каталог для установки программы. По умолчанию используется **C:\Program Files\RedDatabase**.



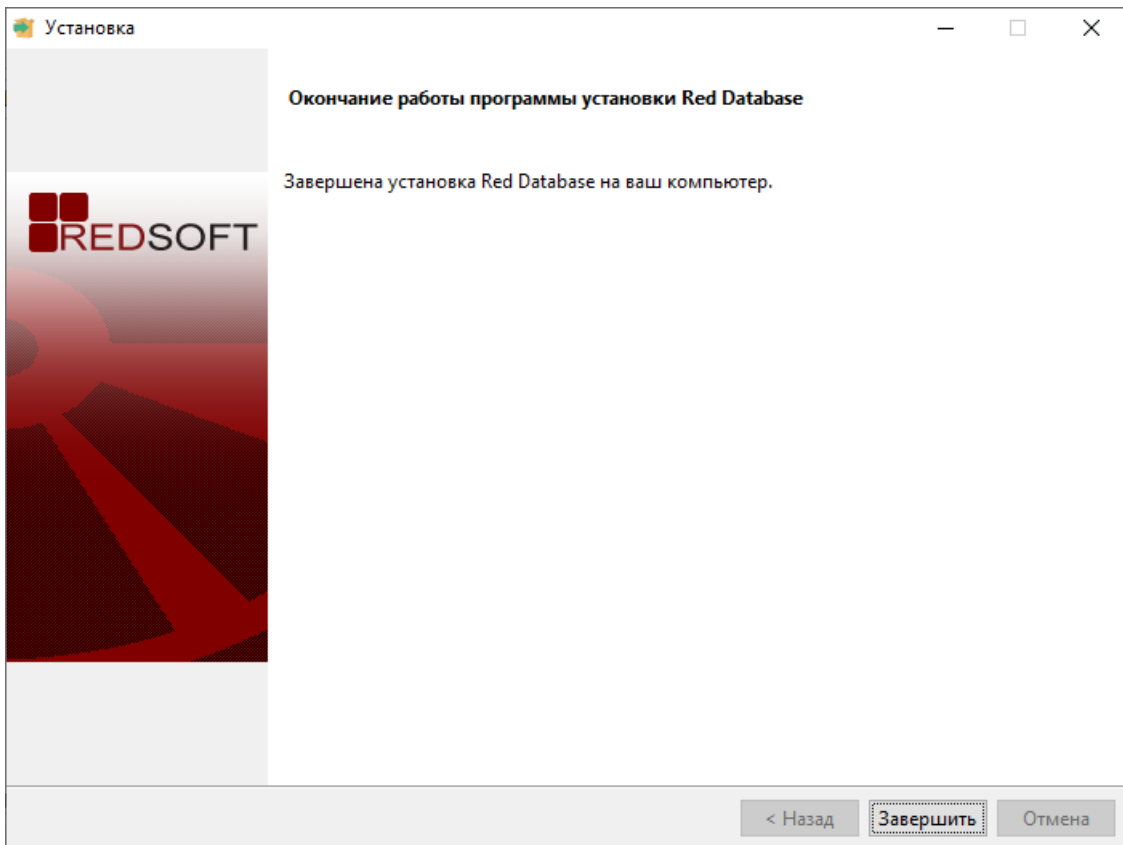
Выберите требуемую архитектуру сервера, установив флаг для соответствующего параметра. Нажмите «**Далее**».



Установите пароль системного администратора. Пароль необходимо повторить для избежания опечаток. Нажмите «**Далее**».

Проверьте указанные ранее параметры. Если все настроено верно, нажмите «**Далее**» для запуска процесса установки. В противном случае нажмите «**Назад**» и внесите изменения в необходимые конфигурации.

После завершения процесса установки будет выведено соответствующее сообщение. Нажмите «**Завершить**».



Обновление СУБД РЕД База Данных

Скачать актуальную версию СУБД РЕД База Данных можно [на официальном сайте](#) компании.

Общий алгоритм обновления выглядит следующим образом:

1. Остановить СУБД Ред База Данных.
2. Проверить отсутствие подключений ко всем файлам баз данных на сервере и отсутствие процессов СУБД.
3. Сделать резервную копию каталога с СУБД (например, переименовав его).
4. Удалить текущую версию СУБД.
5. Выполнить установку новой версии СУБД.
6. Скопировать конфигурационные файлы со старой версии СУБД, дополнительно может потребоваться копирование плагинов (**plugins**) и *java (jar)* библиотек, которые не входят в типовой дистрибутив СУБД и устанавливаются и настраиваются отдельно.

Подробнее с обновлением СУБД можно ознакомиться в [Руководстве администратора](#) на сайте СУБД.

Диагностика проблем

Наличие мусора

Свободное место на диске

Оперативная память

Нагрузка на процессор

Нагрузка на диск

Таблица блокировок

Активность индексов

Поломка БД

Запрос дополнительной информации для первичной диагностики проблемы
при обращении на ЛТП1

Анализ проблем производительности и поломки БД:

- Низкая производительность, симптомы:
 - долгое выполнение запросов;
 - долгое открытие окон в пользовательском интерфейсе;
 - жалобы на работу, вплоть до того, что вообще невозможно подключиться.
- Поломка БД, симптомы:
 - зависание процессов СУБД;
 - аварийное завершение процессов СУБД РБД при выполнении запросов;
 - ошибки в логе **firebird.log**.

Проблема производительности требует анализа на всех уровнях:

- прикладное ПО (толстый клиент, тонкий клиент, сервер приложений);
- сеть (проблемы и ошибки сети, загрузка сети, антивирусы и файерволы);
- гипервизор на сервере (конкуренция с другими ВМ, память, процессор);
- ОС СУБД (постоянное потребление процессора на уровне 100%, очереди к диску, сеть, файловая система);
- проблемы внутри БД (блокировки, мусор, отсутствие индексов, неоптимальные планы).

Все показатели требуется проверять в момент пиковой нагрузки на сервер или зависания и сравнивать с показателями системы, работающей в штатном режиме.

Примеры проведения диагностики проблем

Наличие мусора

Проверку наличия мусора можно осуществить утилитой **gstat**:

```
/opt/RedDatabase/bin/gstat -h /path/to/db.fdb
```

Пример вывода:

```
Database "/opt/db/red.fdb"
Gstat execution time Wed Jun 30 15:53:27 2021

Database header page information:
Flags      0
Generation 236121
System Change Number 24
Page size  16384
Server     RedDatabase
ODS version 12.0
Oldest transaction 234835
Oldest active 234836
Oldest snapshot 234836
Next transaction 234839
Sequence number 0
Next attachment ID 1604
Implementation HW=AMD/Intel/x64 little-endian OS=Linux CC=gcc
Shadow count 0
Page buffers 2048
Next header page 0
Database dialect 3
Creation date Aug 18, 2020 15:08:32
Attributes force write

Variable header data:
Database backup GUID: {C45D8AFE-4293-4EEA-B798-F0360284C9D6}
Sweep interval: 0
Database GUID: {A5D296A8-833A-4377-F496-187F6ABB0D5E}
*END*

Gstat completion time Wed Jun 30 15:53:27 2021
```

Рекомендации:

- Большая разница между **OAT** (Oldest active) и **OIT** (Oldest transaction) — повод для принудительной сборки мусора с помощью команды `gfix -sweep`.
- Большая разница между **Oldest active** и **Next transaction** — долгие транзакции, рекомендуется обратиться к разработчикам прикладного ПО для исключения длинных транзакций. Если такое исключение невозможно, то рекомендуется принудительное завершение таких транзакций (например, каждый день завершать транзакции старше 12-24 часов) и сборка мусора командой `gfix -sweep`.

Свободное место на диске

Проверка наличия свободного места на диске, в том числе в каталоге `/tmp`, производится командой:

```
df -h
```

Рекомендации:

- при отсутствии или недостаточном объеме свободного места необходимо его физическое увеличение либо удаление ненужных файлов.

Оперативная память

Проверка наличия свободной оперативной памяти и занятость **swap** осуществляется командой

```
free -m
```

Использование файла подкачки — плохой признак, который также может быть причиной низкой производительности. Интенсивность использования **swap** можно оценить с помощью утилиты `atop`, см. параметры `swout` и `swin`, их активное изменение говорит о частом использовании **swap**.

Рекомендации:

- проверьте параметры конфигурационного файла **firebird.conf**:
 - `TempBlockSize` — минимальный размер блока сортировки и шаг его

расширения (при необходимости). Позволяет несколько ускорить работу алгоритма сортировки за счёт выделения памяти большими блоками. Определение оптимальных для конкретной задачи настроек может быть произведено экспериментальным путем, принимая во внимание размеры файлов сортировки. При наличии больших сортировок рекомендуется увеличить значение параметра (максимально 2147483647 байт).

- **TempCacheLimit** — для **SuperServer** и **SuperClassic** рекомендуется 64 Мб, для **Classic** = 8 Мб;
- **TempDirectories** — указываются через точку с запятой, следуют в порядке очереди; пока не заполнится первая директория, вторая будет ожидать. Рекомендуется упорядочить каталоги по скорости доступа.

- Настройте использование **swap**:

Создайте файл **/etc/sysctl.d/ram.conf**:

```
nano /etc/sysctl.d/ram.conf
```

Со следующим содержимым:

```
vm.swappiness = 10

vm.dirty_ratio = 60

vm.dirty_background_ratio = 2

vm.min_free_kbytes = 1048576
```

После сохранения изменений выполните перезагрузку операционной системы.

Нагрузка на процессор

Проверка нагрузки на процессор:

```
top (atop, nmon)
```

Длительная 100% загрузка одного ядра — признак зависания.

Рекомендации:

- принудительно завершите этот процесс (в случае с архитектурой **Classic**), предварительно сняв **дамп** для передачи на исследование разработчикам, после чего обратитесь в Техническую поддержку;
- выполните команду **gfix -sweep** без активных подключений к БД;
- проверьте активность индексов и выполните пересчет селективности индексов (рекомендуется выполнять минимум 1 раз в неделю в период минимальной нагрузки);
- низкая загрузка процессоров — ожидание блокировки, смотрите таблицу блокировок (см. п. «[Таблица блокировок](#)»).
- проверьте трейс, возможно необходимо добавление индекса.

Нагрузка на диск

Проверка нагрузки на диск осуществляется командой:

```
iostat -xz (iostat -p sda -x -t 1)
```

где **sda** - имя исследуемого блочного устройства ввода/вывода.

Важное значение имеет длина очереди к диску **aqu-sz** или **avgqu-sz** (название параметра зависит от версии утилиты) — средний размер очереди запросов к диску.

Рекомендации:

- Проверить опции монтирования для **SSD = noatime,discard** (не рекомендуется использовать опцию монтирования **discard** на программном рейде **mdadm** из **NVME SSD**).
- Необходимо понять, какие запросы выполняют наибольшее количество input/output-операций. В этом помогут таблицы мониторинга (**MON\$**) и анализ трейса. Возможно, потребуется создание дополнительного индекса и пересчет селективности индексов (**SET STATISTICS INDEX INDEX_NAME**).

Если на ЭВМ, на котором установлена СУБД РЕД База Данных, используется схема реализации компьютерной памяти **NUMA** (Non-Uniform Memory Access), то работу архитектуры можно проверить с помощью утилиты **numastat**.

В выводе утилиты промахов по памяти (**numa_miss** — сколько раз промахнулись и вынуждены обращаться к памяти другого процессора) должно быть на порядок меньше попаданий (**numa_hit**), в идеале параметр **numa_miss** должен быть равен 0.

Рекомендации:

Если значение `numa_miss` высокое, следует привязать процессы Ред Базы к одному из процессоров (`numactl -C 0 /opt/RedDatabase/bin/rdbserver`), а к другому процессору можно привязать сервер приложений или веб-сервер, если он расположен на том же сервере.

Дополнительно необходимо протестировать производительность дисковой подсистемы с помощью утилиты **fio**:

```
fio fio.conf
```

Пример конфигурационного файла:

```
[ro]

blocksize=16k

directory=/mnt/ssd

rw=randread

direct=1

size=15g

ioengine=psync

iodepth=5

runtime=600

iodepth=5

ioengine=psync
```

[wo]

blocksize=16k

directory=/mnt/ssd

rw=randwrite

direct=1

size=15g

ioengine=psync

iodepth=5

runtime=600

iodepth=5

ioengine=psync

Основные параметры конфигурационного файла:

- блоки **ro** и **wo** (на чтение и на запись) — размер блока 16Кб, можно установить 8Кб, если размер страницы в БД равен 8;
- **directory** — каталог, в котором будет проводиться тестирование;
- **rw=randread** — случайное чтение/запись;
- **direct=1** — флаг **o_direct**;
- **size=100** — размер файла 100Мб (из этого файла производится случайное чтение/запись);
- **runtime=60** — время тестирования;
- **iodepth=5** — глубина очереди к диску, которую **fio** выдает в файл;
- **ioengine=psync** — используемые функции чтения/записи, которыми пишет СУБД.

Параметры, на которые необходимо обратить внимание в выводе теста **fio**:

- **IOPS** — количество операций чтения и записи в секунду;
- **lat** — задержка диска, необходимо ориентироваться на среднее (**avg**) значение.

Таблица блокировок

Примеры использования:

- печать всей таблицы:

```
/opt/RedDatabase/bin/rdb_lock_print -a -d /path/to/db > /path/to/lock_table.txt
```

- печать блокировок с очередями

```
/opt/RedDatabase/bin/rdb_lock_print -n -l -o -d /path/to/db > /path/to/lock_table.txt
```

- поиск зависшего процесса

```
/opt/RedDatabase/bin/rdb_lock_print -q 30 -d /path/to/db
```

В выводе **rdb_lock_print -a -d** можно увидеть следующие параметры:

- **Length** — максимальный размер таблицы блокировок на момент снятия снапшота;
- **Used** — текущий размер таблицы блокировок;
- **Mutex wait** — время ожидания мьютексов в процентах;
- **Hash slots** — ширина хэш-таблицы, которая используется для поиска блокировок. Число должно быть простым, чтобы хэш-алгоритм производил хорошее распределение;
- **Hash length** (*min/avg/max*) — длина хэш-таблицы;
- **Owners** — число соединений к серверу.

Рекомендации:

- Если в пиковые нагрузки в параметре **Hash length** значения **avg** > 3 или **max** > 10, то необходимо увеличивать параметр **LockHashSlots** (в **firebird.conf**), значение должно быть простым числом (максимум 65521).
- Высокий **Mutex wait** считается проблемой, нужно проверить селективность

индексов и их активность.

- Встречается, что все процессы висят на ожидании одной блокировки. Это можно увидеть по выводу команды:

```
/opt/RedDatabase/bin/rdb_lock_print -c -n -l -o -d | grep "Pending"
```

Максимальное число и покажет проблемную блокировку. По типу и ключу можно попытаться понять, какая это блокировка, страница или другой ресурс. Это позволит предоставить рекомендации прикладному разработчику.

- Повисший процесс, который выдаст `rdb_lock_print -q 30`, рекомендуется завершить, предварительно сняв [дамп](#).

Активность индексов

Определить неактивные индексы можно запросом:

```
select rdb$index_name from rdb$indices
where (rdb$system_flag is null or rdb$system_flag = 0)
and RDB$INDEX_INACTIVE > 0
order by rdb$foreign_key nulls first
```

Рекомендации:

- активируйте все индексы:

```
EXECUTE BLOCK AS
DECLARE VARIABLE stmt VARCHAR(1000);
BEGIN
for select 'ALTER INDEX '||rdb$index_name ||' ACTIVE;'
from rdb$indices
where (rdb$system_flag is null or rdb$system_flag = 0)
and RDB$INDEX_INACTIVE > 0
order by rdb$foreign_key nulls first
into :stmt do EXECUTE STATEMENT :stmt;
END
```

- активируйте нужные индексы вручную.

Поломка БД

Поломка БД — это изменения в файле, нарушающие целостность и приводящие к ошибкам.

Основные причины:

- аппаратные (память, диск);
- комплексные (батарейка рейд-контроллера);
- программные (ошибки в коде СУБД).

При поломке БД ошибки могут встречаться как непосредственно при работе пользователей, так и в файле **firebird.log**.

Критические ошибки в файле *firebird.log*:

- internal Firebird consistency check (cannot find tip page);
- Error while trying to read from file;
- Error while trying to write to file;
- internal Firebird consistency check (can't continue after bugcheck);
- internal Firebird consistency check (decompression overran buffer);
- Pointer page (sequence NUM) lost;
- Pointer page (sequence NUM) inconsistent;
- Missing index root page;
- Transaction inventory pages lost;
- Transaction inventory pages confused;
- internal Firebird consistency check (can't continue after bugcheck);
- internal Firebird consistency check (cannot continue after replication failure).

При подозрении на поломку необходимо выполнить проверку файла базы данных через утилиту **gfix**. Для проверки требуется монопольный доступ к базе (при сбоях проверки добавьте ключ **-ignore**):

```
/opt/RedDatabase/bin/gfix -v -full
```

Если ошибки не обнаружены, вывод команды будет пуст.

При обнаружении ошибок вывод команды будет иметь примерно следующий вид:

```
Summary of validation errors
```

```
Number of record level errors : 93
Number of data page errors : 25
Number of index page errors : 29
Number of database page errors : 14
```

Рекомендации:

- выполните проверку:

```
/opt/RedDatabase/bin/gfix -v -full
```

- проверьте лог **firebird.log** на предмет наличия ошибок или отправьте его в службу техподдержки;
- проверьте оборудование (диск, ОЗУ);
- выполните ремонт БД по результатам анализа проверки.

Запрос дополнительной информации для первичной диагностики проблемы при обращении на ЛТП1

Все показатели требуется смотреть в момент пиковой нагрузки на сервер или зависания. Сбор данных необходимо производиться под учетной записью с правами **администратора (sudo)**. Для диагностики необходимо установить следующие пакеты:

```
sudo dnf install numactl sysstat atop htop fio
```

1. Подробное описание проблемы:

- проблема присутствует постоянно или в определенные моменты;
- снижение производительности — общее или при определенных операциях;
- проводились ли какие-либо работы на сервере (обновление, замена оборудования и пр.).

2. Характеристики оборудования (ОЗУ, ЦПУ, hdd или ssd).

3. Вывод команды:

```
df -h
```

4. Вывод команды:

```
sudo gstat -h /path/to/db.fdb
```

5. Вывод команды:

```
sudo rdb_lock_print -l -n -o -d /path/to/db.fdb
```

6. Вывод команды:

```
sudo rdb_lock_print -q 30 -d /path/to/db
```

7. Вывод утилиты **top (atop, htop)** для определения нагрузки на ЦПУ.

8. Вывод команды:

```
free -m
```

9. Вывод утилиты **numastat** (при наличии **NUMA**-архитектуры):

```
numastat
```

10. Вывод утилиты **iostat** для оценки очереди к диску (выполнение займет ~1 минуту):

```
iostat -xzm -t 1 60 > /tmp/iostat.txt
```

При этом сформируется файл **/tmp/iostat.txt**, который необходимо предоставить.

11. Копия **firebird.log**, **firebird.conf**, **databases.conf** и **replication.conf** (при наличии) из каталога установки СУБД.

12. Копия **/var/log/messages** за интервал наблюдения проблемы и в течение месяца до нее.

13. Копия **/etc/fstab**.

14. Вывод команд:

```
sudo dmesg -T > /tmp/dmesg.txt
```

```
sudo journalctl -p err > /tmp/journalctl.txt
```

При этом создадутся файлы **dmesg.txt** и **journalctl.txt** в каталоге **/tmp**.

15. Проверить активность индексов запросом:

```
select rdb$index_name from rdb$indices
where (rdb$system_flag is null or rdb$system_flag = 0)
and RDB$INDEX_INACTIVE > 0
order by rdb$foreign_key nulls first
```

16. Протестировать производительность дисковой подсистемы, на которой расположена БД. Сделать это можно с помощью утилиты **fio**:

```
sudo fio /tmp/fio.conf
```

Файл **fio.conf** необходимо создать с помощью любого текстового редактора, например **nano**:

```
sudo nano /tmp/fio.conf
```

Пример конфигурационного файла:

```
[ro]

blocksize=16k

directory=/mnt/ssd

rw=randread

direct=1

size=15g

ioengine=psync

iodepth=5

runtime=600

iodepth=5
```

ioengine=psync

[wo]

blocksize=16k

directory=/mnt/ssd

rw=randwrite

direct=1

size=15g

ioengine=psync

iodepth=5

runtime=600

iodepth=5

ioengine=psync

Основные параметры конфигурационного файла:

- блоки **ro** и **wo** (на чтение и на запись) — размер блока 16Кб, можно установить 8Кб, если размер страницы в БД равен 8;
- **directory** — каталог, в котором будет проводиться тестирование;
- **rw=randread** — случайное чтение/запись;
- **direct=1** — флаг **o_direct**;
- **size=100** — размер файла 100Мб (из этого файла производится случайное чтение/запись);
- **runtime=60** — время тестирования;
- **iodepth=5** — глубина очереди к диску, которую **fio** выдает в файл;

- `ioengine=psync` — используемые функции чтения/записи, которыми пишет СУБД.

Часто задаваемые вопросы по СУБД РЕД База Данных (FAQ)

+ Где скачать СУБД РЕД База Данных?

Единственным правильным источником для получения сборки является официальный сайт. Дистрибутивы из остальных источников использовать нежелательно, это может быть потенциально опасным.

Официальный источник (он же официальный сайт) — <https://rdb.red-soft.ru/downloads/>.

В разделе «Загрузки» можно увидеть все файлы, доступные для скачивания. Необходимо быть зарегистрированным пользователем. При этом, если учетная запись имеет соответствующие допуски, то список доступных для скачивания продуктов шире.

Чтобы релиз-кандидаты были доступны для скачивания, необходимо в своем профиле установить флаг для параметра «Показывать релиз-кандидаты», в этом случае можно видеть свежие версии, которые, вероятно, в последующем станут релизами. Это может потребоваться, если есть подозрения, что в релиз-кандидате была исправлена какая-либо серьезная ошибка. У каждой версии есть описание, можно просмотреть список изменений и исправленных проблем.

Клиентам выдается учетная запись (логин и пароль) с доступом к приобретенным продуктам.

Основные файлы на странице загрузки:

- **.bin** — установочный файл для ОС семейства Linux;
- **.tar.gz** — архив с файлами СУБД для ОС семейства Linux;
- **.exe** — установочный файл для ОС семейства Windows;
- **.zip** — архив с файлами СУБД для ОС семейства Windows;
- ***dbg-linux-x86_64.tar.gz** — отладочные символы для СУБД, установленной на ОС семейства Linux;
- ***dbg-windows-x86_64.zip** — отладочные символы для СУБД, установленной на ОС семейства Windows.

+ Где скачать документацию на СУБД РЕД База Данных?

Документация доступна на официальном сайте СУБД РЕД Базы Данных — <https://rdb.red-soft.ru/product/docs/>.

Документация регулярно обновляется, исправляется и дополняется при обновлении СУБД, поэтому рекомендуется проверять новые версии документации.

+ Где найти видеоролики по СУБД РЕД База Данных?

Обучающие видеоролики доступны на официальных каналах СУБД РЕД База Данных:

- на **RuTube** — <https://rutube.ru/channel/29879950/>;
- на **Яндекс.Дзен** — <https://dzen.ru/reddatabase>.

+ Какие есть информационные ресурсы в Интернете по СУБД РЕД База Данных?

СУБД Ред База Данных в **Telegram** — <https://t.me/reddatabase>.

Канал СУБД Ред База Данных на **RuTube** — <https://rutube.ru/channel/29879950/>.

Канал СУБД Ред База Данных на **Яндекс.Дзен** — <https://dzen.ru/reddatabase>.

Список наиболее значительных изменений в новых версиях СУБД РЕД База Данных — <https://rdb.red-soft.ru/about/news/>.

Площадка для общения **СУБД РЕД База Данных** и **Firebird** — https://t.me/firebird_reddatabase.

+ Какую архитектуру СУБД выбрать?

Архитектура **Super** — рекомендована к использованию на версии **3.0.9+**. Имеет общий кеш, в версии **3.0** она стала действительно многопоточной, до этого она не была ориентирована на многоядерные процессоры. Страничный кеш общий, что дает увеличение производительности. Однако при аварийном завершении того самого единственного процесса, аварийно прекращает свою работу весь сервер.

Архитектура **Classic** — классическая архитектура, где каждый коннект происходит в отдельном процессе. Каждый процесс имеет собственный кеш данных, что является более надежным решением, но в то же время, в ряде случаев, показывает производительность ниже, чем архитектура **Super**.

Архитектура **Superclassic** — не рекомендована к использованию, в дальнейшем, возможно, будет изъята из дистрибутива.



Чем отличается одна редакция СУБД от другой?

Основные различия описаны на официальном сайте в разделе «Продукты» — <https://rdb.red-soft.ru/product/subd/?tab=2>.



Как установить СУБД на Linux?

Необходимо предоставить права на выполнение. Далее можно выполнять установку в графическом либо в текстовом режиме.

Для назначения прав выполните команду:

```
sudo chmod a+x /path/to/rdb.bin
```

Для установки в графическом режиме выполните:

```
./rdb.bin
```

Далее следуйте инструкциям установщика.

Для установки в текстовом режиме выполните команду:

```
./rdb.bin --mode text
```

Далее следуйте инструкциям установщика.



В чем заключается типовая настройка СУБД после её установки?

Ключевые параметры конфигурационного файла **firebird.conf** для временных файлов:

- **TempTableDirectories** — это каталог, в котором будут создаваться файлы временных таблиц, сейчас они создаются в каталоге **/tmp**.
- **TempDirectories** — список временных каталогов для сортировок, временных таблиц и т. п., разделяются точкой с запятой (;),

используются по порядку. При отсутствии параметра используются переменные окружения `FIREBIRD_TMP`, `TEMP`, `TMP`, поэтому в **systemd** необходимо прописать параметр `FIREBIRD_TMP`. Рекомендуется упорядочить каталоги по скорости доступа, сначала оперативная память, затем **ssd**, затем **hdd**. Если оперативной памяти в избытке, первым можно указать каталог, смонтированный в ОЗУ. Рекомендуется временные каталоги размещать в разделах, отличных от размещения БД. После настройки необходимо следить за состоянием этих каталогов для своевременного исправления возникающих проблем. Пример:

```
TempDirectories = /var/tmp;/mnt/ssd;/mnt/hdd_tmp
```

- **TempCacheLimit** — используется для ограничения объёма временного пространства, которое может быть кешировано в оперативной памяти. В этом пространстве будут сохраняться данные сортировок, буферов записи, небольших временных блоков перед материализацией и т.д. В целях увеличения производительности, желательно установить это значение больше длины временных файлов в каталогах **TempDirectories**, если это позволяет объем доступной оперативной памяти.

Ключевые параметры конфигурационного файла **firebird.conf** для кеширования:

- **DefaultDbCachePages** — количество страниц одной базы данных, находящихся в кеш-памяти одновременно. Супер-сервер использует единый кеш (по умолчанию 2048 страниц), т. е. `DefaultDbCachePages` общий для всех подключений. В архитектуре **Classic** используется свой кеш на каждый процесс, поэтому в архитектуре **Super** значение необходимо устанавливать больше. Измеряется в количестве страниц, поэтому от размера 1 страницы будет зависеть размер памяти, которая будет задействована под кеш.

Предполагается использовать четверть ОЗУ для нужд кеша СУБД.

Значение кеша задается параметром `page buffer` (см. вывод `gstat -h`) для каждой БД в отдельности, либо значением `DefaultDbCachePages` из конфигурационного файла **firebird.conf** для всего сервера, либо параметром `DefaultDbCachePages` в конфигурационном файле **databases.conf**.

В приоритете - параметр `page buffer`, если `page buffer = 0`, то значение берется из конфигурационных файлов.

Архитектура **Super**:

$\text{MemorySize}/(4*\text{PageSize})$

Архитектура **Classic**:

$\text{MemorySize}/(4*\text{ConnNum}*\text{PageSize})$

Настройку можно уточнять по итогу мониторинга сервера: учесть объемы для сортировки, файловый кеш, ядро ОС, другие приложения. Самый важный параметр, на который стоит обратить внимание — это количество чтений (Read) страниц и количество Fetch страниц. **Fetch** — страница берется из кеша (из памяти), если в кеше её нет, то она читается с диска (что замедляет скорость работы). Чем ближе эти значения (write и cache) друг к другу, тем хуже. В идеально прогретом кеше чтений почти не будет, будет только запись — это в архитектуре "супер". В классике кеш у коннектов разный, поэтому чтений там больше. Это связано с необходимостью записи страниц.

- **FileSystemCacheThreshold** — если значение указано больше, чем в **DefaultDBCachePages**, то включается использование файлового кэша. Если значение меньше — отключается. Предопределённых рекомендаций не существует. Подбор оптимального значения параметра зависит от диска и нагрузки, поэтому необходимо наблюдать за использованием диска и подбирать оптимальное значение. Для **Classic** оптимальный параметр обязателен, для **Super** менее критичен.

Далее перечислены ключевые параметры, касающиеся авторизации:

- **AuthServer** и **AuthClient** — набор методов аутентификации, разрешенных на сервере и клиенте:
 - Безопасная парольная (**Srp**) аутентификация;
 - Традиционная (**Legacy_Auth**) аутентификация;
 - Доверительная (**Win_Sspi**) аутентификация для ОС Windows;
 - Многофакторная (**Multifactor**) аутентификация с применением политик безопасности;
 - Доверенная аутентификация через механизм GSSAPI (**Gss**)
- **UserManager** — плагин для работы с базой данных пользователей:
 - Srp;
 - Legacy_UserManager;
 - Multifactor_Manager.
 - Используется первый плагин из списка
- **DefaultUserManagers** — набор стандартных плагинов, в которых по умолчанию будет создаваться пользователь или меняться пароль у пользователя.

Прочие важные параметры **firebird.conf**:

- **ServerMode** — режим работы сервера (**Super, Classic, SuperClassic**), требуется перезапуск при изменении, при смене режима необходимо корректировать параметр кеша.
- **RemoteBindAddress** — позволяет привязать входящие соединения к определенному сетевому интерфейсу. При этом все входящие соединения через другие сетевые интерфейсы будут запрещены.
- **LockAcquireSpins** — количество попыток, которые будут сделаны в условном режиме захвата мьютекса таблицы блокировок. Этот параметр позволяет, вместо обращения к ядру, ждать некую атомарную переменную, что уменьшает количество тактов процессора на межпроцессорные вызовы.
- **LockHashSlots** — число слотов хэширования блокировок. Следует увеличивать если под нагрузкой в таблице блокировок много коллизий. См. позднее `rdb_lock_print -d <database> | <alias>` и `Hash lengths`. Как правило выставляется по максимуму, из опыта.



В чем заключается типовая настройка сервера после установки СУБД?

Для увеличения допустимого числа процессов СУБД и доступных им ресурсов:

- в **/etc/security/limits.conf** лимиты `nproc` и `nofile` для пользователя, от имени которого запускается СУБД (firebird), устанавливаются в 10000;
- параметр ядра `vm.max_map_count` необходимо увеличить до **256000**, написав в **/etc/sysctl.conf** строку:

```
vm.max_map_count=256000
```

Для немедленного применения настройки необходимо выполнить:

```
sysctl -p
```

- для systemd-систем в **/usr/lib/systemd/firebird.service** пропишите:

```
Environment = FIREBIRD_TMP=/path/to/tmp
```

- При наличии достаточного объема **RAM** можно смонтировать в память временный каталог, куда сервер будет выгружать файлы. Для этого в **/etc/fstab** необходимо строку:

```
tmpfs /tmp tmpfs size=32G 0 0
```

где **32G** — максимальный размер, до которого может расшириться временный каталог в памяти;

- отключить **transparent Huge Pages**, например, записав в **/etc/rc.local**:

```
echo never > /sys/kernel/mm/redhat_transparent_hugepage/enabled
```

+ Как понять, какая редакция СУБД Ред База данных у меня установлена?

Необходимо изучить содержимое каталога установки СУБД.

Актуально для **3.0+**:

- если есть **replication.conf** — это промышленная редакция;
- если его нет, но есть **jvm.args** — это стандартная редакция;
- иначе открытая.

Если есть возможность подключиться к любой БД на сервере, то редакцию можно определить, выполнив запрос:

```
select rdb$get_context('SYSTEM', 'EDITION') from rdb$database;
```

+ Как узнать установленную версию СУБД?

Для определения установленной версии СУБД можно воспользоваться утилитой **gbak** с ключом **-z**, например:

```
/opt/RedDatabase/bin/gbak -z
```

Пример вывода команды:

```
gbak:gbak version LI-V3.0.9.0 RedDatabase 3.0 rc.5  
(6dcadeecf66529566457de89e35603e10380fa17)  
gbak: ERROR:requires both input and output filenames  
gbak:Exiting before completion due to errors
```

где **LI-V3.0.9.0 RedDatabase 3.0 rc.5** — версия СУБД.

+ Как обновить СУБД на Linux (минорное обновление)?

Для обновления СУБД на Linux необходимо:

- остановить СУБД и проверить статус службы командой:

```
sudo systemctl stop firebird
systemctl status firebird
```

- Закомментировать алиас в **databases.conf** и через **isql** проверить, что к БД нет подключений:

```
select count(*) from mon$attachments;
exit;
```

- Проверить, что БД никто не использует утилитой **lsof** (от **root** или **sudo**), не должны выводиться pid'ы процессов:

```
sudo lsof /path/to/db.fdb
sudo lsof /opt/RedDatabase/security3.fdb
```

- **П е р е и м е н о в а т ь /opt/RedDatabase** в **/opt/RedDatabase_<старая_версия>**. Старую версию можно узнать например с помощью утилиты **gbak**:

```
/opt/RedDatabase/bin/gbak -z
```

Переименование:

```
sudo mv /opt/RedDatabase /opt/RedDatabase_3.0.5.116
```

- установить новую версию СУБД, используя установочный бинарный файл, для этого назначить необходимые права:

```
sudo chmod a+x /path/to/rdb.bin
```

и запустить установку:

```
./rdb.bin
```

- Из каталога со старой версией СУБД скопировать в новый каталог необходимые конфигурации (**directories.conf**, **databases.conf**, **firebird.conf**, **fbtrace_sec.conf**, **fbtrace_dba.conf**, **replication.conf**, **fbjava.yaml**), дополнительные плагины (если такие имеются) и базы данных безопасности (**security3.fdb**, **java-security.fdb**);
- раскомментировать алиас базы в **databases.conf**:
- запустить РЕД Базу Данных и проверить статус службы (должно быть **active (running)**);

```
sudo systemctl start firebird  
systemctl status firebird
```

- проверить успешность подключения к базе по сетевому протоколу через **isql** (полный путь до файла БД):

```
/opt/RedDatabase/bin/isql -u sysdba -p masterkey localhost:alias
```

+ Как проверить БД на наличие ошибок?

Проверить БД на наличие ошибок можно утилитой **gfix**, требуется монопольный доступ к БД:

```
/opt/RedDatabase/bin/gfix -v -full /path/to/database.fdb
```

gfix может попытаться исправить некоторые не критичные ошибки. Для отмены исправления не критичных ошибок необходимо добавить ключ **-n**:

```
/opt/RedDatabase/bin/gfix -v -full -n /path/to/database.fdb
```

+ Какие ошибки можно считать критичными при проверке?

Все ошибки файла БД после проверки фиксируются в логе **/opt/RedDatabase/firebird.log**. Список наиболее критичных из них:

- **internal Firebird consistency check (cannot find tip page)** — нарушение некоторой целостности, например, не получается найти страницу, на которой отмечен статус транзакции, или находится не та страница;

- **Error while trying to {read from, write to} file** — не может прочитать/записать файл БД;
- **internal Firebird consistency check (decompression overran buffer)** — например, при декомпрессии какой-либо записи обнаруживается, что она превышает размер положенного буфера;
- **Pointer page (sequence NUM) {lost, inconsistent}** — потеряна страница указателей;
- **Missing index root page** — потеря **root page** для индекса;
- **Transaction inventory pages lost** — потерянная страница транзакций;
- **internal Firebird consistency check (cannot continue after replication failure)**.

+ В чем заключается типовой ремонт базы данных?

Типовой ремонт заключается в процедуре резервного копирования (**gbak -b -v -g**) и восстановления (**gbak -c -v -o**) поврежденной базы данных.

Ни в коем случае нельзя удалять поврежденную БД до полного завершения ремонта.

+ Что делать с ошибкой **invalid request BLR** при восстановлении резервной копии?

В данном случае можно попробовать перекомпилировать проблемные процедуры и еще раз сделать резервную копию. Если используется СУБД 3.0 и выше, восстановить такую резервную копию можно с ключом **-ig**, который восстановит процедуру с пустым **BLR**. Далее её необходимо перекомпилировать из исходников.

+ Что делать, если процесс создания резервной копии или процесс ее восстановления аварийно завершается при прохождении данных из определенной таблицы?

В данном случае БД можно восстановить с пропуском данных проблемной таблицы (или нескольких таблиц). Для этого используется ключ **-skip_d**. Пример команды:

```
/opt/RedDatabase/bin/gbak -user SYSDBA -pas masterkey -b -v -g -skip_d
'(TABLE_NAME_1|TABLE_NAME_2)' /path/to/database.fdb /path/to/backup.fbk -y
/path/to/backup.log
```

При этом нужно помнить, что деактивируются внешние ключи, которые касаются таблиц с пропущенными данными. После восстановления БД таким способом необходимо загрузить недостающие данные и активировать вручную внешние ключи.

+ Как проверить наличие мусора в БД?

Утилитой `gstat` с ключом `-h`:

```
/opt/RedDatabase/bin/gstat -h /path/to/database.fdb
```

В выводе нужно обратить внимание на маркеры:

```
Oldest transaction, Oldest active, Oldest snapshot, Next transaction.
```

Разница между `Oldest transaction` и `Oldest snapshot` должна стремиться к единице. Однако зачастую разница этих параметров достигает больших значений, что негативно влияет на производительность. Для решения данной проблемы рекомендуется регулярная (ежедневная) сборка мусора.

+ Как принудительно собрать мусор в БД?

Принудительный сбор мусора в БД производится утилитой `gfix`:

```
/opt/RedDatabase/bin/gfix -sweep /path/to/database.fdb
```

+ Как сделать резервную копию базы данных?

Утилитой `gbak`, стандартная команда выглядит следующим образом:

```
/opt/RedDatabase/bin/gbak -user SYSDBA -pas masterkey -b -v -g  
/path/to/database.fdb /path/to/backup.fbk -y /path/to/backup.log
```

Для того чтобы убедиться, что процесс создания резервного копирования успешно завершился, необходимо проверить его лог, формирование которого определяется ключом `-y` (см. команду создания резервной копии) — `/path/to/backup.log`. При успешном завершении лог заканчивается следующими строками:

```
gbak:writing names mapping
gbak:closing file, committing, and finishing. 82432 bytes written
gbak:stopped at Mon Aug 22 11:37:08 2022
```

При аварийном завершении или наличии ошибок следует разбираться в проблеме предметно.

+ Как восстановить резервную копию БД?

Восстановление резервной копии БД производится утилитой **gbak**. Стандартная команда выглядит следующим образом:

```
/opt/RedDatabase/bin/gbak -user SYSDBA -pas masterkey -c -v -o
/path/to/backup.fbk /path/to/database.fdb -y /path/to/restore.log
```

При необходимости нужно использовать ключ **-ig** — при ошибках с восстановлением **BLR** хранимых процедур и функций процесс создания резервной копии продолжается, при этом в логе останется запись о пропуске **BLR**.

Для того чтобы убедиться, что восстановление резервной копии успешно завершено, следует проверить его лог, формирование которого определяется ключом **-y** (см. команду восстановления резервной копии) — **/path/to/restore.log**.

При успешном завершении лог заканчивается следующими строками:

```
gbak:fixing views dbkey length
gbak:updating ownership of packages, procedures and tables
gbak:adding missing privileges
gbak:adjusting system generators
gbak:finishing, closing, and going home
gbak:adjusting the ONLINE and FORCED WRITES flags
gbak:stopped at Mon Aug 22 11:52:21 2022
```

БД переводится в ONLINE-режим.

При аварийном завершении или наличии ошибок следует разбираться в проблеме предметно.

+ данных и для чего они нужны?

Отладочные символы — специальные данные о связи адресов и исходных кодов, некие описания разных переменных, какая инструкция к какой строке кода относится. Нужны для анализа зависимостей и получения стеков, чтобы разработчик смог посмотреть работу процесса в виде текстовой информации, которую он видит у себя в процессе разработки.

Для установки необходимо скачать нужный архив с официального сайта. В названии архива присутствует обозначение **dbg** и версия отладочных символов, например, **RedDatabase-5.0.7-dbg-linux-x86_64.tar.gz**.

ВАЖНО!

Версия СУБД и версия символов должны строго совпадать.

Далее архив необходимо распаковать в нужное место. Для этого:

- перейдите в **mc**;
- перейдите в папку с отладочными символами;
- откройте архив как обычную директорию;
- в подкаталогах из архива существуют скрытые папки **debug**, в которых как раз и находятся все отладочные символы. Скопируйте все каталоги из архива в директорию **RedDatabase** (каждый каталог из архива в соответствующий ему каталог в /opt/RedDatabase).

На этом установка закончена.

+ Как настроить сбор дампов на Linux?

Необходимо использовать **systemd-coredump** для автоматического сбора дампов:

https://redos.red-soft.ru/base/manual/utilites/coredump/?sphrase_id=285533

+ Что такое репликация и как её настроить?

Репликация — непрерывное копирование (реплицирование) данных (изменений этих данных) с одного сервера (*master*) на один или несколько других (*slave*, реплика, зеркало).

В СУБД Ред База Данных реализовано два вида логической репликации:

- **Синхронная** — обеспечивает прямое подключение к *slave* и отправку изменений на него непосредственно до коммита транзакции;
- **Асинхронная** — предполагает запись в промежуточный журнал, который передается на зеркало и там применяется.

Примеры настройки репликации описаны [Руководстве администратора](#) в подразделе "Настройка системы репликации".

+ Как проверить работу асинхронной репликации?

Утилита **rdblogmgr** – запускать на мастере:

```
sudo rdblogmgr -u sysdba -p masterkey -d /db/replicated.fdb
```

В выводе можно увидеть следующие параметры:

- *current sequence* – текущий сегмент;
- *last modified* – последнее изменение;
- *total log size* – всего в логе байт в 1 сегменте;
- *free segments* – свободных сегментов 1, полных сегментов 0, архивных 0.

Пример вывода:

Log status:

Current sequence: 8

Last modified: 2023-12-13 16:48:19

Total log size: 946259 bytes in 2 segments

Free segments: 2, full segments: 0, archived segments: 0

Утилита **rdbreplmgr** – запускать на зеркале с ключом **-s**:

```
sudo rdbreplmgr -u sysdba -p masterkey /db/replica.fdb -s
```

В выводе команды можно увидеть:

- *Control file* — контрольный файл, содержит текущие счетчики пролитых логов, сколько осталось, сколько активных транзакций (бывает так, что в одном сегменте транзакция стартует, а в другом завершается, и в итоге сегмент можно удалить, когда все транзакции

в нем завершены), при переинициализации его нужно удалять;

- *Current segment* — количество обработанных сегментов, должно постоянно расти;
- *Oldest segment* — количество старых сегментов, должно быть в значении **absent**;
- *Oldest segment* — в идеале **absent**;
- Total segments in the queue — количество сегментов в очереди на обработку, в идеале должно стремиться к 0.

Пример вывода:

```
Status for replica /opt/db/red.fdb:
Master database: 192.168.1.10:/opt/db/red.fdb
Master GUID: {71615D98-0551-48D3-EE94-9441AFF3FE04}
Archive directory: /mnt/repl/logsrdb/arch/
Control file: /mnt/repl/logsrdb/arch/{71615D98-0551-48D3-EE94-9441AFF3FE04}
Current segment: 16 (as of 2020-06-17 09:36:35)
Oldest segment: absent
Total segments in the queue: 1
```



Почему при использовании репликации важно наличие первичных ключей в таблице?

Если на мастере создали таблицу без первичного ключа и уникального индекса, то репликационные журналы перестают вливаться на зеркале, при этом отображается ошибка вида:

```
my-server (slave) Tue Dec 22 11:24:35 2020
Database: /path/to/database.fdb
ERROR: Table MY_TABLE has no unique key
```

В конфигурационном файле **replication.conf** на мастере необходимо прописать параметр **exclude_without_pk = true**, при этом таблицы без первичного ключа не будут реплицироваться на базу зеркала.



Что такое трейс и как его запустить?

Трейс (аудит) — инструмент для аудита событий, происходящих внутри БД, позволяет просматривать:

- подключения (отключения) к БД;
- все стадии выполнения запросов, включая счетчики производительности;
- события безопасности;
- работа со службами;
- выполнение хранимых процедур, функций и триггеров;
- работа транзакций, параметры транзакций и многое другое.

Информации может быть много, поэтому в конфигурационном файле доступны различные фильтры (например, по подключению или по времени). Аудит можно сделать на уровне сервера (системный), а можно открыть интерактивную сессию аудита.

- системный аудит — запуск происходит с помощью раскомментирования и изменения параметра `enabled` в конфигурационном файле **fbtrace.conf** — `enabled = true`;
- интерактивный сеанс аудита — запуск происходит с помощью утилиты **rdbtracemgr** и заранее подготовленного конфигурационного файла.

```
rdbtracemgr <параметры_подключения> <действия>
[<параметры>]
```

Пример команды:

```
/opt/RedDatabase/bin/rdbtracemgr -SE service_mgr -START -NAME my_trace -
CONFIG my_cfg.txt
```

+ Как найти неактивные индексы?

Для этого необходимо подключиться к базе данных и выполнить SQL-запрос в любом менеджере БД:

```
select rdb$index_name from rdb$indices where (rdb$system_flag is null or
rdb$system_flag = 0) and RDB$INDEX_INACTIVE > 0;
```

+ Как активировать все неактивные индексы?

С помощью SQL-запроса в любом менеджере БД:

```
EXECUTE BLOCK AS
DECLARE VARIABLE stmt VARCHAR (1000);
BEGIN
for select 'ALTER INDEX '||rdb$index_name ||' ACTIVE;'
from rdb$indices
where (rdb$system_flag is null or rdb$system_flag = 0)
and RDB$INDEX_INACTIVE > 0
order by rdb$foreign_key nulls first
into :stmt do EXECUTE STATEMENT :stmt;
END
```

+ Как получить структуру индексов определенной таблицы?

С помощью SQL-запроса в любом менеджере БД:

```
SELECT S.RDB$INDEX_NAME, S.RDB$FIELD_NAME FROM RDB$INDICES I
NATURAL JOIN RDB$INDEX_SEGMENTS S WHERE
I.RDB$RELATION_NAME='MYTABLE' ORDER BY S.RDB$FIELD_POSITION
```

где вместо 'MYTABLE' необходимо указать имя нужной таблицы.

+ Как получить список ролей или пользователей, которые имеют права на вставку в таблицу?

С помощью SQL-запроса в любом менеджере БД:

```
SELECT RDB$USER FROM RDB$USER_PRIVILEGES WHERE
RDB$RELATION_NAME='COUNTRY' AND RDB$PRIVILEGE='I' AND
RDB$USER_TYPE=13 AND RDB$OBJECT_TYPE=0
```

+ Как найти все подключения старше суток?

С помощью SQL-запроса в любом менеджере БД:

```
SELECT MON$ATTACHMENT_ID FROM MON$ATTACHMENTS
```

```
WHERE DATEDIFF (HOUR, MON$TIMESTAMP, CURRENT_TIMESTAMP) > 24
```

Для их принудительного завершения:

```
DELETE FROM MON$ATTACHMENTS  
WHERE DATEDIFF (HOUR, MON$TIMESTAMP, CURRENT_TIMESTAMP) > 24
```

+ Как принудительно завершить подключения с транзакциями старше суток?

С помощью SQL-запроса в любом менеджере БД:

```
DELETE FROM MON$ATTACHMENTS  
WHERE MON$ATTACHMENT_ID IN (  
SELECT MON$ATTACHMENT_ID FROM MON$TRANSACTIONS  
WHERE DATEDIFF (HOUR, MON$TIMESTAMP, CURRENT_TIMESTAMP) > 24)
```

+ Как принудительно завершить соединения с запросами старше суток?

С помощью SQL-запроса в любом менеджере БД:

```
DELETE FROM MON$ATTACHMENTS  
WHERE MON$ATTACHMENT_ID IN (  
SELECT MON$ATTACHMENT_ID FROM MON$STATEMENTS  
WHERE DATEDIFF (HOUR, MON$TIMESTAMP, CURRENT_TIMESTAMP) > 24)
```

+ Как найти самые читающие с диска запросы?

С помощью SQL-запроса в любом менеджере БД:

```
SELECT MON$SQL_TEXT FROM MON$STATEMENTS  
WHERE MON$STAT_ID = (SELECT FIRST 1 MON$STAT_ID FROM MON$IO_STATS  
WHERE MON$STAT_GROUP=3 ORDER BY MON$PAGE_READS DESC)
```

Для их завершения:

```
DELETE FROM MON$ATTACHMENTS
WHERE MON$STAT_ID = (SELECT FIRST 1 MON$STAT_ID FROM MON$IO_STATS
WHERE MON$STAT_GROUP=1 ORDER BY MON$PAGE_READS DESC)
MON$RECORD_STATS
```

+ Как найти запросы, которые потребляют больше всего памяти?

С помощью SQL-запроса в любом менеджере БД:

```
SELECT MON$SQL_TEXT FROM MON$STATEMENTS
WHERE MON$STAT_ID = (SELECT FIRST 1 MON$STAT_ID FROM
MON$MEMORY_USAGE WHERE MON$STAT_GROUP=3 ORDER BY
MON$MEMORY_USED DESC)
```

+ Как найти клиентские подключения, которые потребляют больше всего памяти?

С помощью SQL-запроса в любом менеджере БД:

```
SELECT MON$REMOTE_ADDRESS, MON$REMOTE_PROCESS
FROM MON$ATTACHMENTS
WHERE MON$STAT_ID = (SELECT FIRST 1 MON$STAT_ID FROM
MON$MEMORY_USAGE WHERE MON$STAT_GROUP=1 ORDER BY
MON$MEMORY_USED DESC)
```

+ Какие утилиты доступны "из коробки" для работы с файлом базы данных?

Для этих целей можно использовать утилиту **isql**. С помощью нее можно создавать базу данных, подключаться и выполнять SQL-запросы в базе данных.

Доступна в любой редакции СУБД, входит в состав дистрибутива.

ISQL — это утилита командной строки для работы с базами данных РЕД

Базы Данных при помощи языка структурированных запросов (Structured Query Language — SQL).

Пример команды для подключения:

```
/opt/RedDatabase/bin/isql -u sysdba -p masterkey localhost:employee
```

где:

- *sysdba* — имя пользователя, от которого подключаемся к БД;
- *masterkey* — пароль этого пользователя;
- *employee* — алиас базы данных.

Подробное описание возможностей данной утилиты доступно в [Руководстве администратора](#) в разделе «**Утилита ISQL**».



Почему нужно следить за наличием свободного места на разделе с базой данных?

Нехватка места на разделе с базой данных чревата поломкой самой базы, поэтому рекомендуется на разделе иметь хотя бы 30% свободного места (минимум 10%). При этом в логе могут появляться ошибки вида:

```
my.server.local (master) Sun Sep 20 04:43:29 2020
Database: /path/to/database.fdb
ERROR: Log file /path/to/repl/logs/database.fdb.log-142 write failed (error 28)

my.server.local (master) Sun Sep 20 04:43:29 2020
Database: /path/to/database.fdb
WARNING: Slave is to be detached after error. Replica may become
inconsistent at this point.
```



Что нужно отслеживать на сервере СУБД?

1. Наличие мусора и выполнение регулярной сборки мусора.

2. Наличие свободного места в ключевых разделах:

- раздел с БД;
- в корневом разделе;
- в разделе с установленной СУБД;

- наличие свободного места в каталоге **/tmp**.

3. Загрузку ОЗУ (параметр **available** в выводе **free -m**).
4. Лог **firebird.log** (сообщения об ошибках, остановках сервиса).
5. Системный лог (сообщения об аварийном завершении).
6. Проверка лога создания резервной копии и тестового восстановления резервной копии.
7. Проверка наличия долгих транзакций.

+ Как посмотреть размер объектов в БД?

Для проверки размера объектов в БД необходимо собрать статистику по файлу базы данных с помощью команды:

```
/opt/RedDatabase/bin/gstat -user sysdba -password masterkey -a -r -s  
my_database.fdb > /home/user/Загрузки/full_gstat.txt
```

По окончании выполнения команды будет сформирован файл **/home/user/Загрузки/full_gstat.txt**, используемый далее для подсчета размера объектов.

Пример вывода статистики:

```
Database "/databases/my_base.fdb"  
Gstat execution time Fri Dec 22 11:15:11 2023
```

Database header page information:

Flags	0
Generation	164
System Change Number	0
Page size	8192
Server	RedDatabase
ODS version	12.3
Oldest transaction	126
Oldest active	127
Oldest snapshot	127
Next transaction	127
Autosweep gap	1
Sequence number	0
Next attachment ID	27

```
Implementation      HW=AMD/Intel/x64 little-endian OS=Linux CC=gcc
Shadow count        0
Page buffers         0
Next header page     0
Database dialect     3
Creation date        Dec 22, 2023 10:36:15
Attributes           force write
```

Variable header data:

```
Sweep interval:      20000
Database GUID: {EA77BD7A-868B-4459-F5A2-3E578634467F}
*END*
```

...

DOCUMENT (132)

```
Primary pointer page: 204, Index root page: 205
Total formats: 1, used formats: 1
Average record length: 524.10, total records: 1000000
Average version length: 0.00, total versions: 0, max versions: 0
Average fragment length: 0.00, total fragments: 0, max fragments: 0
Average unpacked length: 1010.00, compression ratio: 1.93
Pointer pages: 45, data page slots: 72056
Data pages: 72056, average fill: 92%
Primary pages: 72056, secondary pages: 0, swept pages: 0
Empty pages: 3, full pages: 72052
Fill distribution:
  0 - 19% = 3
 20 - 39% = 0
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 72053
```

Для определения размера таблицы в выводе следует обратить внимание на параметр **Data pages**, затем умножить количество **Data pages** нужной таблицы на размер страницы базы данных (размер страницы определяется параметром **Page size** из заголовка базы данных). В результате будет получен размер данных таблицы в байтах без учета блобов и индексов.

Например, таблица **DOCUMENT** по статистике имеет **72056 страниц**, размер одной страницы — **8192 Б**. Размер таблицы равен $72056 \times 8192 = \sim 563$ МБ, среднее заполнение 92%.

Размер блобов

Для определения размера блобов в выводе следует обратить внимание на

строку **Blobs**, содержащую значение параметра **total length**. Если в таблице есть блобы, то в статистике таблицы будет присутствовать следующая строка:

```
Blobs: 1289124, total length: 12577030488, blob pages: 863868
```

Здесь параметр **total length** равен **12577030488** — т.е. примерно 11,7 ГБ.

Размер индексов

Для определения размера индексов следует обратить внимание на значение параметра **leaf buckets**, затем умножить значение параметра **leaf buckets** на размер страницы базы данных (размер страницы определяется параметром **Page size** из заголовка базы данных). В результате будет получен размер индекса. Например:

```
Index RDB$PRIMARY27 (0)
  Root page: 72963, depth: 3, leaf buckets: 1205, nodes: 1000000
  Average node length: 9.71, total dup: 0, max dup: 0
  Average key length: 6.72, compression ratio: 1.00
  Average prefix length: 3.01, average data length: 3.74
  Clustering factor: 999982, ratio: 1.00
  Fill distribution:
    0 - 19% = 1
    20 - 39% = 0
    40 - 59% = 0
    60 - 79% = 0
    80 - 99% = 1204
```

Здесь параметр **leaf buckets** имеет размер **1205 страниц**, размер одной страницы (из статистики) — **8192 Б**. Размер индекса равен $1205 \times 8192 = \sim 9,4$ МБ.

Соответствие стандарту SQL:2023

На странице представлены таблицы соответствия функционала РЕД Базы Данных обязательным и опциональным возможностям стандарта SQL:2023.

Обозначения:

- "+" - поддерживается;
- "-" - не поддерживается;
- "*" - частично поддерживается.

Обязательные возможности:

Идентификатор	Возможность	RedDatabase 5
E011	Числовые типы данных	+
E011-01	Типы данных INTEGER и SMALLINT (все варианты написания)	+
E011-02	Типы данных REAL, DOUBLE PRECISION и FLOAT	+
E011-03	Типы данных DECIMAL и NUMERIC	+
E011-04	Арифметические операторы	+
E011-05	Числовые сравнения	+
E011-06	Неявные преобразования между числовыми типами данных	+
E021	Символьные типы данных	+
E021-01	Тип данных CHARACTER (все варианты написания)	+
E021-02	Тип данных CHARACTER VARYING (все варианты написания)	+
E021-03	Символьные литералы	+
E021-04	Функция CHARACTER_LENGTH	+
E021-05	Функция OCTET_LENGTH	+
E021-06	Функция SUBSTRING	+
E021-07	Конкатенация символов	+
E021-08	Функции UPPER и LOWER	+
E021-09	Функция TRIM	+

E021-10	Неявное преобразование между типами символьных строк фиксированной и переменной длины	+
E021-11	Функция POSITION	+
E021-12	Сравнение символов	+
E031	Идентификаторы	+
E031-01	Идентификаторы с разделителями	+
E031-02	Идентификаторы в нижнем регистре	+
E031-03	Завершающее подчёркивание	+
E051	Базовое определение запросов	*
E051-01	SELECT DISTINCT	+
E051-02	Предложение GROUP BY	+
E051-04	GROUP BY может содержать столбцы не из <списка выборки>	+
E051-05	Элементы списка выбора могут быть переименованы	+
E051-06	Предложение HAVING	+
E051-07	Квалифицированный * в списке выбора	+
E051-08	Корреляционные имена в предложении FROM	+
E051-09	Переименование столбцов в предложении FROM	*
E061	Базовые предикаты и условия поиска	+
E061-01	Предикат сравнения	+
E061-02	Предикат BETWEEN	+
E061-03	Предикат IN со списком значений	+
E061-04	Предикат LIKE	+
E061-05	LIKE predicate: ESCAPE clause	+
E061-06	Предикат NULL	+
E061-07	Предикат количественного сравнения	+
E061-08	Предикат EXISTS	+
E061-09	Подзапросы в предикате сравнения	+
E061-11	Подзапросы в предикате IN	+
E061-12	Подзапросы в количественном предикате сравнения	+

E061-13	Коррелированные подзапросы	+
E061-14	Условие поиска	+
E071	Простые выражения с запросами	*
E071-01	Табличный оператор UNION DISTINCT	+
E071-02	Табличный оператор UNION ALL	+
E071-03	Табличный оператор EXCEPT DISTINCT	-
E071-05	Столбцы, объединяемые табличными операторами, могут иметь разные типы данных	+
E071-06	Табличные операторы в подзапросах	+
E081	Основные привилегии	+
E081-01	Привилегия SELECT на уровне таблицы	+
E081-02	Привилегия DELETE	+
E081-03	Привилегия INSERT на уровне таблицы	+
E081-04	Привилегия UPDATE на уровне таблицы	+
E081-05	Привилегия UPDATE на уровне столбца	+
E081-06	Привилегия REFERENCES на уровне таблицы	+
E081-07	Привилегия REFERENCES на уровне столбца	+
E081-08	WITH GRANT OPTION	+
E081-09	Привилегия USAGE	+
E081-10	Привилегия EXECUTE	+
E091	Set functions	+
E091-01	AVG	+
E091-02	COUNT	+
E091-03	MAX	+
E091-04	MIN	+
E091-05	SUM	+
E091-06	ALL квантификатор	+
E091-07	DISTINCT квантификатор	+
E101	Основные манипуляции с данными	+
E101-01	Оператор INSERT	+

E101-03	Оператор UPDATE с критерием отбора	+
E101-04	Оператор DELETE с критерием отбора	+
E111	Оператор SELECT, возвращающий одну строку	+
E121	Базовая поддержка курсоров	*
E121-01	DECLARE CURSOR	+
E121-02	Столбцы ORDER BY, отсутствующие в списке выборки	+
E121-03	Выражения значений в предложении ORDER BY	+
E121-04	Оператор OPEN	+
E121-06	Оператор UPDATE с позиционированием	+
E121-07	Оператор DELETE с позиционированием	+
E121-08	Оператор CLOSE	+
E121-10	Оператор FETCH с неявным NEXT	+
E121-17	Курсоры WITH HOLD	-
E131	Поддержка значений NULL (NULL вместо значений)	+
E141	Основные ограничения целостности	*
E141-01	Ограничения NOT NULL	+
E141-02	Ограничения UNIQUE столбцов NOT NULL	+
E141-03	Ограничения PRIMARY KEY	+
E141-04	Базовое ограничение FOREIGN KEY с NO ACTION по умолчанию и для операций удаления со ссылками, и для операций изменения со ссылками	+
E141-06	Ограничения CHECK	+
E141-07	Значения столбцов по умолчанию	+
E141-08	NOT NULL распространяется на PRIMARY KEY	+
E141-10	Имена во внешнем ключе могут определяться в любом порядке	-
E151	Поддержка транзакций	+
E151-01	Оператор COMMIT	+
E151-02	Оператор ROLLBACK	+
E152	Базовый оператор SET TRANSACTION	*
E152-01	Оператор SET TRANSACTION: предложение ISOLATION LEVEL SERIALIZABLE	-

E152-02	Оператор SET TRANSACTION: предложения READ ONLY и READ WRITE	+
E153	Обновляемые запросы с подзапросами	+
E161	Комментарии SQL с использованием двойного минуса в начале	+
E171	Поддержка SQLSTATE	+
E182	Привязки для системных языков	-
F031	Основные манипуляции со схемой	*
F031-01	Оператор CREATE TABLE для создания постоянных базовых таблиц	+
F031-02	Оператор CREATE VIEW	+
F031-03	Оператор GRANT	+
F031-04	Оператор ALTER TABLE: предложение ADD COLUMN	+
F031-13	Оператор DROP TABLE: предложение RESTRICT	*
F031-16	Оператор DROP VIEW: предложение RESTRICT	*
F031-19	Оператор REVOKE: предложение RESTRICT	*
F041	Базовое соединение таблиц	+
F041-01	Внутреннее соединение (но не обязательно с ключевым словом INNER)	+
F041-02	Ключевое слово INNER	+
F041-03	LEFT OUTER JOIN	+
F041-04	RIGHT OUTER JOIN	+
F041-05	Внешние соединения могут быть вложенными	+
F041-07	Внутренняя таблица в левом или правом внешнем соединении также может использоваться во внутреннем соединении	+
F041-08	Поддерживаются все операторы сравнения (не только =)	+
F051	Базовая поддержка даты и времени	+
F051-01	Тип данных DATE (включая поддержку строк DATE)	+
F051-02	TIME data type (including support of TIME literal) with fractional seconds precision of at least 0	+

F051-03	Тип данных TIME (включая поддержку литерала TIME) с точностью до долей секунды не менее 0	+
F051-04	Предикаты сравнения с типами данных DATE, TIME и TIMESTAMP	+
F051-05	Явный CAST между типами даты/времени и типами символьных строк	+
F051-06	CURRENT_DATE	+
F051-07	LOCALTIME	+
F051-08	LOCALTIMESTAMP	+
F081	UNION и EXCEPT в представлениях	*
F131	Операции группировки	+
F131-01	Предложения WHERE, GROUP BY и HAVING, поддерживаемые в запросах со сгруппированными представлениями	+
F131-02	Поддержка нескольких таблиц в запросах со сгруппированными представлениями	+
F131-03	Поддержка функций множеств в запросах со сгруппированными представлениями	+
F131-04	Подзапросы с предложениями GROUP BY и HAVING и сгруппированные представления	+
F131-05	SELECT, возвращающий одну строку, с предложениями GROUP BY и HAVING и сгруппированными представлениями	+
F181	Поддержка множества модулей	-
F201	Функция CAST	+
F221	Явные значения по умолчанию	*
F261	Выражение CASE	+
F261-01	Простой CASE	+
F261-02	Оператор CASE с условиями	+
F261-03	NULLIF	+
F261-04	COALESCE	+
F311	Оператор определения схемы	-
F311-01	CREATE SCHEMA	-
F311-02	CREATE TABLE для хранимых основных таблиц	-
F311-03	CREATE VIEW	-

F311-04	CREATE VIEW: WITH CHECK OPTION	-
F311-05	Оператор GRANT	-
F471	Скалярные значения подзапросов	+
F481	Расширенный предикат NULL	+
S011	Отдельные типы данных	-
T321	Основные процедуры, вызываемые SQL	*
T321-01	Пользовательские функции без перегрузки	+
T321-02	Пользовательские хранимые процедуры без перегрузки	+
T321-03	Вызов функций	+
T321-04	Оператор CALL	*
T321-05	Оператор RETURN	+
T631	Предикат IN с одним элементом списка	+

Опциональные возможности:

Идентификатор	Возможность	RedDatabase 5
B011	Встроенный язык Ada	+
B012	Встроенный C	+
B013	Встроенный язык COBOL	+
B014	Встроенный язык Fortran	+
B015	Встроенный язык MUMPS	-
B016	Встроенный язык Pascal	+
B017	Встроенный язык PL/I	-
B021	Непосредственный SQL	+
B030	Расширенный динамический SQL	-
B031	Базовый динамический SQL	*
B032	Улучшенный динамический SQL	-
B036	Оператор describe input	-
B033	Нетипизированные аргументы функции, вызываемой SQL	-

B034	Динамическое указание атрибутов курсора	-
B035	Нерасширенные имена дескрипторов	-
B041	Расширения встроенных объявлений исключений SQL	-
B051	Улучшенные права на выполнение	-
B111	Язык модулей — Ada	-
B112	Язык модулей — C	-
B113	Язык модулей — COBOL	-
B114	Язык модулей — Fortran	-
B115	Язык модулей — MUMPS	-
B116	Язык модулей — Pascal	-
B117	Язык модулей — PL/I	-
B121	Язык подпрограмм — Ada	+
B122	Язык подпрограмм — C	+
B123	Язык подпрограмм — COBOL	+
B124	Язык подпрограмм — Fortran	+
B125	Язык подпрограмм — MUMPS	+
B126	Язык подпрограмм — Pascal	+
B127	Язык подпрограмм — PL/I	+
B128	Язык подпрограмм — SQL	+
B200	Полиморфные табличные функции (PTF)	-
B201	Более одного абстрактного табличного параметра PTF	-
B202	Соразделение PTF	-
B203	Более одного указания соразделения	-
B204	PRUNE WHEN EMPTY	-
B205	Сквозные столбцы	-
B206	Передача в PTF параметров с дескриптором	-
B207	Векторное произведение разделов	-
B208	Интерфейс компонентных процедур PTF	-
B209	Расширенные имена PTF	-

B211	Язык модулей — Ada: поддержка VARCHAR и NUMERIC	+
B221	Язык подпрограмм — Ada: поддержка VARCHAR и NUMERIC	-
F032	Каскадное удаление (CASCADE)	-
F033	Оператор ALTER TABLE: предложение DROP COLUMN	+
F034	Расширенный оператор REVOKE	*
F035	REVOKE с CASCADE	-
F036	Оператор REVOKE, выполняемый не владельцем	+
F037	Оператор REVOKE: предложение GRANT OPTION FOR	+
F038	REVOKE с привилегией WITH GRANT OPTION	+
F052	Арифметика с интервалами и датами/временем	*
F053	Предикат OVERLAPS	-
F054	TIMESTAMP в списке приоритетов типа DATE	-
F111	Уровни изоляции, отличные от SERIALIZABLE	*
F112	Уровень изоляции READ UNCOMMITTED	-
F113	Уровень изоляции READ COMMITTED	+
F114	Уровень изоляции REPEATABLE READ	+
F121	Базовое управление диагностикой	-
F120	Оператор GET DIAGNOSTICS	-
F124	Оператор SET TRANSACTION: предложение DIAGNOSTICS SIZE	-
F122	Расширенное управление диагностикой	-
F123	Вся диагностика	-
F171	Несколько схем для одного пользователя	-
F191	Действия при удалении со ссылками	+
F200	Оператор TRUNCATE TABLE	-
F202	TRUNCATE TABLE: возможность перезапуска столбца идентификации	-
F222	Оператор INSERT: предложение DEFAULT VALUES	+
F251	Поддержка доменов	+
F262	Расширенные выражения CASE	+

F263	Разделённые запятыми предикаты в простом выражении CASE	-
F271	Составные символьные строки	-
F281	Улучшенный оператор LIKE	+
F291	Предикат UNIQUE	-
F292	Обработка NULL в ограничениях UNIQUE	-
F301	CORRESPONDING в выражениях запросов	-
F302	Табличный оператор INTERSECT	-
F303	Табличный оператор INTERSECT DISTINCT	-
F305	Табличный оператор INTERSECT ALL	-
F304	Табличный оператор EXCEPT ALL	-
F312	Оператор MERGE	+
F313	Улучшенный оператор MERGE	+
F314	Оператор MERGE с ветвью DELETE	+
F321	Авторизация пользователей	*
F361	Поддержка подпрограмм	+
F381	Расширенные манипуляции со схемой	+
F387	Оператор ALTER TABLE: предложение ALTER COLUMN	+
F388	Оператор ALTER TABLE: предложение ADD/DROP CONSTRAINT	+
F382	Изменение типа данных столбцов	*
F383	Предложение, устанавливающее NOT NULL для столбца	+
F384	Предложение удаления свойства идентифицирующего столбца	+
F385	Предложение удаления выражения, генерирующего значения столбца	-
F386	Предложение установления генерирования значений идентифицирующего столбца	+
F391	Длинные идентификаторы	+
F392	Спецсимволы Unicode в идентификаторах	-
F393	Спецсимволы Unicode в текстовых строках	-
F394	Необязательное указание нормальной формы	-

F401	Расширенное соединение таблиц	+
F405	NATURAL JOIN	+
F406	FULL OUTER JOIN	+
F407	CROSS JOIN	+
F402	Соединения по именам столбцов для больших объектов, массивов и мультимножеств	-
F403	Секционированные соединённые таблицы	-
F404	Переменная диапазона для общих имен столбцов	+
F411	Определение часовых поясов	+
F421	Национальные символы	+
F431	Прокручиваемые курсоры только для чтения	+
F432	FETCH с явным NEXT	+
F433	FETCH FIRST	+
F434	FETCH LAST	+
F435	FETCH PRIOR	+
F436	FETCH ABSOLUTE	+
F437	FETCH RELATIVE	+
F438	Прокручиваемые курсоры	+
F441	Расширенная поддержка функций множеств	+
F442	Смешанные ссылки на столбцы в функциях множеств	+
F451	Определение набора символов	-
F461	Именованные наборы символов	+
F491	Управление ограничениями	+
F492	Необязательное указание соблюдения ограничения таблицы	-
F521	Утверждения	-
F531	Временные таблицы	*
F555	Дополнительная точность в секундах	*
F561	Полные выражения значений	+
F571	Проверки значений истинности	+

F591	Производные таблицы	+
F611	Типы данных для индикаторов	-
F641	Конструкторы строк и таблиц	-
F651	Квалификаторы имен каталогов	-
F661	Простые таблицы	-
F671	Подзапросы в ограничениях CHECK	+
F672	Ретроспективные ограничения CHECK	+
F673	Вызов подпрограммы, читающей SQL-данные, в условиях CHECK	+
F690	Поддержка правил сортировки	+
F692	Расширенная поддержка правил сортировки	+
F693	Правила сортировки символов для SQL-сеансов и клиентских модулей	-
F695	Поддержка перекодировки	-
F701	Действия при обновлении со ссылками	+
F711	ALTER для домена	+
F721	Отсроченные ограничения	-
F731	Привилегия INSERT для столбцов	-
F741	Ссылочные типы	-
F751	Улучшение CHECK для представлений	-
F761	Управление сессией	-
F762	CURRENT_CATALOG	-
F763	CURRENT_SCHEMA	-
F771	Управление соединением	-
F781	Самоссылающиеся операции	+
F791	Нечувствительные курсоры	-
F801	Полные функции множеств	-
F821	Ссылки на локальные таблицы	-
F831	Полное обновление курсора	-
F832	Обновляемые прокручиваемые курсоры	-

F833	Обновляемые упорядоченные курсоры	-
F841	Предикат LIKE_REGEX	+
F842	Функция OCCURRENCES_REGEX	-
F843	Функция POSITION_REGEX	-
F844	SUBSTRING_REGEX	+
F845	TRANSLATE_REGEX	-
F846	Поддержка октетов в операторах регулярных выражений	-
F847	Неконстантные регулярные выражения	-
F850	Предложение ORDER BY на верхнем уровне в выражении запроса	+
F851	ORDER BY в подзапросах	+
F852	Предложение ORDER BY на верхнем уровне в представлении	+
F855	Вложенное предложение ORDER BY в выражении запроса	+
F856	Вложенное предложение FETCH FIRST в выражении запроса	+
F857	Предложение FETCH FIRST на верхнем уровне в выражении запроса	+
F858	FETCH FIRST в подзапросах	+
F859	Предложение FETCH FIRST на верхнем уровне в представлениях	+
F860	Динамический подсчёт в предложении FETCH FIRST	+
F861	Предложение OFFSET на верхнем уровне в выражении запроса	+
F862	Предложение OFFSET в подзапросах	+
F863	Вложенное предложение OFFSET в выражении запроса	+
F864	Предложение FETCH FIRST на верхнем уровне в представлениях	+
F865	Динамический подсчёт строк в предложении OFFSET	+
F866	Предложение FETCH FIRST: параметр PERCENT	-
F867	Предложение FETCH FIRST: параметр WITH TIES	-
F868	ORDER BY в сгруппированной таблице	-

R010	Распознавание шаблона строк: предложение FROM	-
R020	Распознавание шаблона строк: предложение WINDOW	-
R030	Распознавание шаблона строк: полная поддержка агрегатов	-
S023	Базовые структурированные типы	-
S024	Расширенные структурированные типы	-
S025	Окончательные структурированные типы	-
S026	Самоссылающиеся структурированные типы	-
S027	Создание метода по заданному имени метода	-
S028	Произвольный порядок параметров UDT	-
S041	Базовые ссылочные типы	-
S043	Расширенные ссылочные типы	-
S051	Создание таблицы из типа	-
S071	SQL-пути при разрешении имён функций и типов	-
S081	Подтаблицы	-
S090	Минимальная поддержка массивов	*
S091	Базовая поддержка массивов	*
S093	Массивы отдельных типов	-
S099	Выражения с массивами	-
S092	Массивы пользовательских типов	-
S094	Массивы ссылочных типов	-
S095	Конструкторы массивов из запроса	-
S096	Необязательное определение границ массива	-
S097	Присваивание значения элементу массива	-
S098	ARRAY_AGG	-
S111	ONLY в выражениях запросов	-
S151	Предикат типа	-
S161	Приведение подтипов	-
S162	Приведение подтипов для ссылочных типов	-

S201	Вызываемые из SQL подпрограммы, работающие с массивами	-
S203	Массивы в параметрах	-
S204	Массивы в качестве типа результата функций	-
S202	Вызываемые из SQL подпрограммы, работающие с мультимножествами	-
S211	Пользовательские функции приведений	-
S231	Указатели на структурные типы	-
S232	Указатели на массивы	-
S233	Указатели на мультимножества	-
S241	Функции преобразований	-
S242	Оператор изменения преобразования	-
S251	Пользовательские упорядочения	-
S261	Метод определения типа	-
S271	Базовая поддержка мультимножеств	-
S272	Мультимножества пользовательских типов	-
S274	Мультимножества ссылочных типов	-
S275	Расширенная поддержка мультимножеств	-
S281	Типы вложенных коллекций	-
S291	Ограничение уникальности для всей строки	-
S301	Расширенный UNNEST	-
S401	Отдельные типы на базе типов массивов	-
S402	Отдельные типы на базе типов мультимножеств	-
S403	ARRAY_MAX_CARDINALITY	-
S404	TRIM_ARRAY	-
T021	Типы данных BINARY и VARBINARY	+
T022	Расширенная поддержка типов данных BINARY и VARBINARY	+
T023	Составные двоичные строки	-
T024	Пробелы в двоичных строках	+
T031	Тип данных BOOLEAN	+

T041	Базовая поддержка типа данных LOB	*
T039	Указатель на CLOB: неудерживаемый	-
T040	Конкатенация типов данных CLOB	+
T045	Тип данных BLOB	+
T046	Тип данных CLOB	*
T047	POSITION, OCTET_LENGTH, TRIM и SUBSTRING для BLOB	+
T048	Конкатенация типов данных BLOB	+
T049	Указатель на BLOB: неудерживаемый	-
T050	POSITION, CHAR_LENGTH, OCTET_LENGTH, LOWER, TRIM, UPPER и SUBSTRING для CLOBs	+
T042	Расширенная поддержка типа данных LOB	-
T043	Множитель T	-
T044	Множитель P	-
T051	Типы кортежей	-
T053	Явные псевдонимы ссылки на все поля	-
T054	GREATEST и LEAST	-
T055	Функции дополнения строк	+
T056	Многосимвольный TRIM	+
T061	Поддержка UCS	-
T062	Единицы длины символа	-
T071	Тип данных BIGINT	+
T076	Тип данных DECFLOAT	+
T081	Необязательное указание максимальной длины типов строк	-
T101	Улучшенное определение возможности NULL	-
T111	Обновляемые соединения, объединения и столбцы	-
T121	WITH (без RECURSIVE) в выражении запроса	+
T122	WITH (с RECURSIVE) в подзапросе	-
T131	Рекурсивный запрос	+
T132	Рекурсивный запрос в подзапросе	-

T133	Расширенные значения метки циклов	-
T141	Предикат SIMILAR	+
T151	Предикат DISTINCT	+
T152	Предикат DISTINCT с отрицанием	+
T171	Предложение LIKE в определении таблицы	-
T172	Предложение подзапроса AS в определении таблицы	-
T173	Расширенное предложение LIKE в определении таблицы	-
T174	Идентифицирующие столбцы	+
T175	Генерируемые столбцы	+
T176	Поддержка генераторов последовательностей	+
T177	Поддержка генераторов последовательностей: возможность простого перезапуска	+
T178	Идентифицирующие столбцы: возможность простого перезапуска	+
T180	Системное версионирование таблиц	-
T181	Таблицы с периодом времени прикладного уровня	-
T191	Действие RESTRICT при нарушении ссылок	+
T200	Триггер DDL	*
T201	Сравнимые типы данных для ссылочных ограничений	*
T211	Базовые возможности триггеров	*
T212	Расширенные возможности триггеров	-
T213	Триггеры INSTEAD OF	-
T214	Триггеры BEFORE	+
T215	Триггеры AFTER	+
T216	Возможность задавать условие поиска, которое должно быть истинным перед вызовом триггера	-
T217	Привилегия TRIGGER	-
T218	Несколько триггеров для одного события вызываются в порядке создания	*
T231	Чувствительные курсоры	-
T241	Оператор SET TRANSACTION	+
T251	Оператор SET TRANSACTION: параметр LOCAL	-

T261	Сцепленные транзакции	-
T262	Мультисерверные транзакции	-
T271	Точки сохранения	+
T272	Улучшенное управление точками сохранения	-
T281	Привилегия SELECT на уровне столбцов	-
T285	Улучшения имён производных столбцов	+
T301	Функциональные зависимости	+
T312	Функция OVERLAY	-
T323	Явное управление безопасностью внешних подпрограмм	-
T324	Явное управление безопасностью подпрограмм SQL	-
T325	Дополненные определения параметров SQL	-
T326	Табличные функции	-
T331	Базовые роли	+
T332	Расширенные роли	*
T341	Перегрузка вызываемых из SQL функций и процедур	-
T351	Блочные комментарии (/*...*/ комментариев)	+
T431	Расширенные возможности группирования	-
T432	Вложения и конкатенация GROUPING SETS	-
T433	Функция GROUPING с несколькими аргументами	-
T434	GROUP BY DISTINCT	-
T441	Функции ABS и MOD	+
T461	Симметричный предикат BETWEEN	-
T471	Наборы результатов в качестве возвращаемого значения	-
T472	DESCRIBE CURSOR	-
T491	Производная таблица LATERAL	+
T495	Совместное изменение и извлечение данных	-
T501	Улучшенный предикат EXISTS	+
T502	Предикаты периодов	-

T511	Счётчики транзакций	-
T521	Именованные аргументы в операторе CALL	+
T522	Значения по умолчанию для входных параметров процедур, вызываемых из SQL	+
T523	Значения по умолчанию для INOUT-параметров процедур, вызываемых из SQL	-
T524	Именованные аргументы при вызове подпрограмм не с применением оператора CALL	-
T525	Значения по умолчанию для параметров функций, вызываемых из SQL	+
T551	Необязательные ключевые слова для синтаксиса по умолчанию	+
T561	Удерживаемые указатели	-
T571	Внешние вызываемые из SQL функции, возвращающие массивы	-
T572	Внешние вызываемые из SQL функции, возвращающие мультимножества	-
T581	Функция подстроки по регулярному выражению	+
T591	Ограничения UNIQUE для столбцов, принимающих NULL	+
T601	Ссылки на локальные курсоры	-
T611	Элементарные операции OLAP	-
T612	Расширенные операции OLAP	-
T613	Получение выборки	-
T614	Функция NTILE	+
T615	Функции LEAD и LAG	+
T616	Варианты обработки NULL для функций LEAD и LAG	-
T617	Функции FIRST_VALUE и LAST_VALUE	+
T618	Функция NTH_VALUE	+
T619	Вложенные оконные функции	-
T620	Предложение WINDOW: параметр GROUPSn	-
T621	Дополнительные численные функции	+
T622	Тригонометрические функции	+
T623	Общие логарифмические функции	+

T624	Расширенные логарифмические функции	+
T625	LISTAGG	*
T626	ANY_VALUE	-
T627	COUNT DISTINCT в рамке окна	-
T641	Присваивание нескольким столбцам	-
T651	Операторы модификации схемы SQL в SQL-подпрограммах	-
T652	Операторы динамического SQL в SQL-подпрограммах	+
T653	Операторы модификации схемы SQL во внешних подпрограммах	-
T654	Операторы динамического SQL во внешних подпрограммах	+
T655	Циклически зависимые подпрограммы	+
T661	Недесятичные целочисленные константы	+
T662	Подчёркивания в числовых литералах	-
T670	Совместное использование операторов изменения данных и схем	-
T801	Тип данных JSON	-
T802	Расширенный тип данных JSON	-
T803	JSON на основе строк	+
T811	Базовые функции-конструкторы SQL/JSON	+
T812	SQL/JSON: JSON_OBJECTAGG	+
T813	SQL/JSON: JSON_ARRAYAGG с ORDER BY	-
T814	Двоеточие в JSON_OBJECT или JSON_OBJECTAGG	+
T821	Основные операторы запросов SQL/JSON	+
T822	SQL/JSON: предикат IS JSON WITH UNIQUE KEYS	+
T823	SQL/JSON: предложение PASSING	+
T824	JSON_TABLE: предложение PLAN	+
T825	SQL/JSON: ON EMPTY и ON ERROR	+
T826	Произвольные выражения значений в предложениях ON ERROR и ON EMPTY	+

T827	JSON_TABLE: одноуровневые предложения NESTED COLUMNS	+
T828	JSON_QUERY	+
T829	JSON_QUERY: возможности обёртывания массивов	+
T830	Требование уникальности ключей в функциях-конструкторах SQL/JSON	+
T831	Язык путей SQL/JSON: строгий режим	*
T832	Язык путей SQL/JSON: методы элементов	+
T833	Язык путей SQL/JSON: множественные индексы	+
T834	Язык путей SQL/JSON: обращение к элементам по звёздочке	+
T835	Язык путей SQL/JSON: выражения фильтров	+
T836	Язык путей SQL/JSON: предикат начала	+
T837	Язык путей SQL/JSON: предикат regex_like	+
T838	JSON_TABLE: предложение PLAN DEFAULT	+
T839	Преобразование даты/времени из символьных строк с форматированием и обратно	+
T840	Целые шестнадцатеричные литералы в языке путей SQL/JSON	-
T851	SQL/JSON: необязательные ключевые слова, подразумеваемые синтаксисом по умолчанию	-
T860	Упрощённое обращение к SQL/JSON: только ссылки на столбцы	-
T861	Упрощённое обращение к SQL/JSON: обращение к элементу JSON с учётом регистра	+
T862	Упрощённое обращение к SQL/JSON: оператор обращения по звёздочке	+
T863	Упрощённое обращение к SQL/JSON: строковый литерал в одинарных кавычках в качестве оператора обращения	+
T864	Упрощённое обращение к SQL/JSON	-
T865	Метод элементов SQL/JSON: bigint()	-
T866	Метод элементов SQL/JSON: boolean()	-
T867	Метод элементов SQL/JSON: date()	-
T868	Метод элементов SQL/JSON: decimal()	-

T869	Метод элементов SQL/JSON: decimal() с точностью и масштабом	-
T870	Метод элементов SQL/JSON: integer()	-
T871	Метод элементов SQL/JSON: number()	-
T872	Метод элементов SQL/JSON:string()	-
T873	Метод элементов SQL/JSON: time()	-
T874	Метод элементов SQL/JSON: time_tz()	-
T875	Метод элементов SQL/JSON: time с точностью	-
T876	Метод элементов SQL/JSON: timestamp()	-
T877	Метод элементов SQL/JSON: timestamp_tz()	-
T878	Метод элементов SQL/JSON: timestamp с точностью	-
T879	JSON в операциях равенства	-
T880	JSON в операциях группировки	-
T881	JSON в операциях упорядочивания	-
T882	JSON в операциях группировки элементов мультимножеств	-